

Содержание	
Введение.....	5
1 Обзор предметной области	7
1.1 Что такое стеганография?	7
1.2 В каких случаях стеганография представляет угрозу.....	9
1.3 Обзор ПО для выявления скрытой информации в изображении	13
Выводы по разделу 1	17
2 Обзор методов стегоанализа.....	19
2.1 Стегоанализ	19
2.2 Статистические методы:	20
2.3 Методы машинного обучения:	24
2.4 Глубокие нейронные сети	25
2.5 Сравнение методов стегоанализа	30
Выводы по разделу 2	34
3 Разработка системы для выявления вставок.....	35
3.1 Архитектура системы	35
3.2 Модель нейронной сети	36
3.3 Датасет	38
Выводы по разделу 3	41
4 Реализация и тестирование системы выявления вставок	42
4.1 Описание среды разработки, обучение модели машинного обучения.	42
4.2 Нужные метрики, анализ эффективности обучения выбранной модели машинного обучения.	43
4.3 Тестирование RS-метода и модели машинного обучения в совокупности	47

Выводы по разделу 4	50
Заключение	51
Список используемых источников	52
ПРИЛОЖЕНИЕ А	54
ПРИЛОЖЕНИЕ Б.....	55
ПРИЛОЖЕНИЕ В	68
ПРИЛОЖЕНИЕ Г	79
ПРИЛОЖЕНИЕ Д	82

Введение

В современном мире информация является одним из самых ценных ресурсов, и её защита становится всё более актуальной задачей. Одним из способов защиты информации является стеганография — наука о скрытой передаче данных. Стеганография позволяет скрывать информацию в различных контейнерах, таких как изображения, аудио- и видеофайлы, делая её незаметной для посторонних глаз.

В контексте растущих угроз, связанных со скрытой передачей информации. Стеганография становится все более изощренной и распространенной в использовании злоумышленниками для скрытой передачи конфиденциальных данных, в том числе и содержимого, которое может нанести какой-либо вред. Разработка эффективных методов обнаружения факта присутствия стеганографических вставок необходима для защиты информационных систем и обеспечения цифровой безопасности.

Стеганография представляет собой метод скрытой передачи информации, который может быть использован как для легальных, так и для противоправных целей. Ниже приведены угрозы, реализуемые с применением стеганографии:

1. Хищение информации.
2. Скрытое внедрение вредоносного ПО для проведения кибератак.
3. Организация скрытых каналов связи при подготовке к совершению преступлений.

В целом, стеганография может быть использована для различных угроз, и важно принимать соответствующие меры для обеспечения безопасности и защиты от потенциальных рисков.

Для обнаружения стеганографических вставок используются различные методы стегоанализа. Эти методы включают сигнатурные, статистические подходы, методы машинного обучения и глубокие нейронные сети. Каждый из этих методов имеет свои преимущества и недостатки.

В данной работе рассматривается разработка системы выявления стеганографических вставок в цифровом контейнере. Будет предложена новая система выявления вставок.

Цель данной работы является разработка системы, которая используется для анализа изображений, чтобы обнаружить внедренную информацию.

Объект исследования в данной дипломной работе — методы обнаружения внедрения информации в изображениях.

Предмет исследования - разработка системы анализа изображения для обнаружения встроенной информации.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Исследование способов применения стеганографии.
2. Поиск и анализ существующего ПО для выявления скрытой информации.
3. Обзор методов стегоанализа.
4. Разработка системы выявления стеганографических вставок.
5. Анализ эффективности системы.

1 Обзор предметной области

1.1 Что такое стеганография?

Стеганография [1] — это способ спрятать информацию внутри другой информации или физического объекта так, чтобы ее нельзя было обнаружить. С помощью стеганографии можно спрятать практически любой цифровой контент, включая тексты, изображения, аудио- и видеофайлы. А когда эта спрятанная информация поступает к адресату, ее извлекают.

В стеганографии контейнером называют любой объект, который служит для незаметного встраивания информации. В качестве контейнеров могут выступать различные цифровые объекты: текстовые документы, аудио- и видеофайлы, изображения и др. Выбор контейнера определяется целями и задачами стеганографии, а также требуемым уровнем скрытности.

Существуют разные способы использования контейнеров для сокрытия информации. Например, в изображениях применяется метод замены наименьшего значащего бита (LSB), при котором скрытое сообщение встраивается в наименее важные биты пикселей изображения [2]. В аудиофайлах используется фазовая модуляция, позволяющая изменить фазу сигнала так, чтобы это было незаметно на слух, но при этом содержало скрытую информацию.

Стеганографические алгоритмы внедрения информации можно разделить на три основных типа:

1. Внедрение в область значений — это метод, при котором скрытое сообщение встраивается в область значений контейнера, например, в изображение, аудио или видеофайл. Это может быть достигнуто путём изменения младших битов пикселей изображения, изменения амплитуды аудиосигнала или временных интервалов видеосигнала [2].
2. Внедрение в область преобразований — это метод, основанный на использовании определённых преобразований контейнера, таких как дискретное косинусное преобразование (DCT) для изображений или

дискретное вейвлет-преобразование (DWT) для аудио и видео, также может использоваться дискретное преобразование Фурье (DFT). Скрытое сообщение встраивается в коэффициенты преобразованного сигнала, которые затем восстанавливаются на принимающей стороне.

3. Внедрение в служебные поля — это метод, при котором скрытое сообщение встраивается в служебные поля файлов различных форматов, такие как заголовки файлов, временные метки или комментарии. Эти поля обычно не видны пользователю и предназначены для использования программами или операционной системой.

Применение стеганографии

Стеганография находит применение в самых разных сферах: от защиты авторских прав и личных данных до секретной коммуникации и обмена засекреченной информацией.

Примеры использования стеганографии:

1. Обход цензуры: новостные сообщения рассылаются в обход цензуры и без опасения, что источник сообщения будет раскрыт.
2. Защита информации: используется правоохранительными и государственными органами для тайной передачи третьим лицам особо важной информации.
3. Защиту авторских прав, то есть наносят цифровые водяные знаки. С помощью стеганографии можно защитить авторские права на цифровые произведения, внося специальные метки в файлы, которые остаются невидимыми для пользователей, но распознаются специальными программами. Это помогает предотвратить незаконное копирование и распространение контента.
4. Передачу секретной информации. Стеганография может использоваться для передачи секретной информации, которая должна оставаться незамеченной. Это может включать в себя передачу военных планов или других критически важных данных.

5. Подтверждение подлинности документов. Стеганография может быть использована для встраивания уникальных идентификаторов в документы, что позволяет подтвердить их подлинность и избежать подделки.

6. Предотвращение утечки информации. Внедрение скрытых меток в документы или электронные письма может служить предупреждением о том, что информация является конфиденциальной и не должна распространяться без разрешения.

7. Скрытую передачу управляющих сигналов. В некоторых случаях стеганография может использоваться для скрытой передачи управляющих сигналов, например, в системах безопасности или военной технике.

8. Подтверждение достоверности переданной информации. Стеганография может использоваться для встраивания цифровых отпечатков в документы или электронные письма, что позволяет подтвердить их подлинность и избежать подделки.

9. Индивидуальный отпечаток в системе электронного документооборота. Стеганография может использоваться для создания индивидуального отпечатка каждого документа в системе электронного документооборота, что позволяет отслеживать его историю и предотвращать несанкционированные изменения.

Важно отметить, что неправильное использование стеганографии может привести к серьёзным последствиям, включая нарушение авторских прав и сокрытие незаконной информации. Поэтому важно ответственно подходить к применению этого метода и использовать его только в законных целях.

1.2 В каких случаях стеганография представляет угрозу.

Использование стеганографии может представлять угрозу в современном мире, так как она позволяет скрыто передавать информацию, делая ее незаметной для сторонних лиц. Это может быть использовано для тайного обмена данными, шпионажа и других незаконных действий.

Угрозы, реализуемые с применением стеганографии:

1. Хищение информации.
2. Скрытое внедрение вредоносного ПО для проведения кибератак.
3. Организация скрытых каналов связи при подготовке к совершению преступлений.

Терроризм:

Террористы могут использовать стеганографию, чтобы скрыть свои коммуникации и планы от правительственных служб безопасности. Путем внедрения сообщений с террористическим содержанием в видео-, аудио- или изображениях они могут общаться и координировать действия, не рискуя быть обнаруженными. Это делает борьбу с терроризмом более сложной, так как террористы могут оперировать в тайне, не попадая под наблюдение правоохранительных органов. Решительные меры по борьбе с использованием стеганографии террористами требуются для обеспечения безопасности общества и предотвращения терактов.

Пример:

Один из способов применения стеганографии – это внедрение информации в изображения, которые затем могут быть отправлены через мессенджеры.

Такой подход делает обнаружение и извлечение секретной информации намного сложнее для спецслужб, поскольку обычно подобные сообщения могут быть прочитаны только с помощью специальных программ и алгоритмов.

Поэтому в борьбе с терроризмом важно учитывать возможность использования стеганографии в мессенджерах и разрабатывать меры по предотвращению таких методов обмена информацией. Ведь благодаря этому террористы могут планировать атаки и координировать свои действия, оставаясь незамеченными.

Сотрудники компании:

Угроза безопасности компании связана с тем, что сотрудники могут использовать стеганографию для передачи конфиденциальной информации

или совершения незаконных действий. Путем скрытого внедрения данных в обычные файлы или сообщения они могут обмениваться информацией, не вызывая подозрений у систем безопасности компании. Это может привести к утечкам важных данных, нарушениям безопасности или финансовым потерям. Необходимо принимать меры по обнаружению и предотвращению использования стеганографии сотрудниками, чтобы защитить компанию от потенциальных угроз и сохранить конфиденциальность информации.

Пример

В Нью-Йорке сотрудник крупной корпорации похитил файлы, содержащие коммерческие тайны своей компании, и внедрил их в пейзажной картинке.

По версии следствия, обвиняемый использовал методы стеганографии, чтобы спрятать украденные данные в невинную на вид картинку с изображением заката. После этого он отправил изображение по электронной почте.

Кибератаки:

Злоумышленники могут применять стеганографию, чтобы внедрить вредоносные данные в файлы, которые кажутся безвредными. Это позволяет им обходить традиционные механизмы обнаружения вирусов и других угроз, делая их атаки более скрытыми и эффективными. Использование стеганографии требует значительных усилий и навыков, поэтому к ней обычно прибегают опытные злоумышленники для достижения конкретных целей.

Применение стеганографии в кибератаках:

1. Встраивание вредоносной нагрузки в цифровые медиафайлы: злоумышленники могут использовать избыточность данных в графических файлах, видео, аудио и документах для скрытия вредоносного кода.

2. Программы-вымогатели и извлечение данных: стеганография позволяет преступникам незаметно извлекать конфиденциальные данные, маскируя их под обычную переписку.

3. Скрытые команды внутри веб-страниц: злоумышленники могут прятать команды для своих имплантов в пустых полях веб-страниц, журналах отладки на форумах и других местах.

4. Вредоносное рекламное ПО: стеганография используется для встраивания вредоносного кода в рекламные баннеры, которые активируются при загрузке и перенаправляют пользователей на страницы с эксплойтами.

Примеры:

Скимминг в интернет-магазинах

В 2020 году специалисты компании Sansec, которая занимается вопросами безопасности электронной торговли, обнаружили, что мошенники добавили на платёжные страницы интернет-магазинов специальные программы для считывания данных банковских карт (скиммеры), спрятав их в файлах векторной графики SVG.

Декодер находился в других элементах веб-страниц, а вредоносная программа была скрыта в векторных изображениях. Пользователи, которые вводили платёжную информацию на таких страницах, не видели ничего подозрительного, так как эти изображения были похожи на логотипы известных компаний.

Случай SolarWinds

В 2020 году злоумышленники внедрили вредоносное программное обеспечение в обновление от SolarWinds — компании, разрабатывающей платформу для управления ИТ-инфраструктурой. Им удалось взломать системы Microsoft, Intel, Cisco и различных государственных учреждений США. Затем с помощью стеганографии они спрятали похищаемую информацию в безобидных XML-файлах, которые передавались в HTTP-ответах от серверов управления.

Промышленные предприятия

В 2020 году хакеры использовали стеганографические документы для атаки на многие предприятия в Великобритании, Германии, Италии и Японии. Они разместили на таких фото-хостингах, как Imgur, изображения, которые

могли заражать Excel-документы. Скрипт, встроенный в изображение, загружал вредоносную программу Mimikatz, которая позволяла злоумышленникам получать пароли Windows.

1.3 Обзор ПО для выявления скрытой информации в изображении

В настоящее время существует ограниченное количество программного обеспечения, которое доступно широкой публике и позволяет выявлять скрытые данные в изображениях. Ниже мы рассмотрим некоторые известные программы в этой области.

Stegdetect [3] – это утилита, предназначенная для анализа изображений на предмет наличия в них стеганографического содержимого. Она использует статистические тесты для определения, было ли в изображение скрыто дополнительное сообщение. Stegdetect также пытается идентифицировать метод, который был использован для внедрения скрытого контента.

Приложение обнаруживает различные методы стеганографии, включая LSB, маскирование и фильтрацию, а также модификации на основе палитры. Программа создаёт отчёт об обнаруженной скрытой информации, уровне достоверности и расположении изменений в изображении. Также она может анализировать каталог изображений.

Есть недостатки. Программа может ошибочно принять не стеганографические модификации за скрытую информацию. Она бесполезна против продвинутых методов стеганографии. Приложение работает только с JPEG-изображениями и не имеет удобного интерфейса. Программа является свободно распространяемым ПО с открытым исходным кодом.

Stego Suite [4] - комплексное программное решение, предназначенное для стеганографического анализа и защиты данных. Оно включает в себя два основных модуля: Stego Analyst и Stego Break.

Stego Analyst – это визуальный аналитический пакет, предназначенный для анализа цифровых изображений и аудиофайлов. С его помощью

пользователи могут исследовать файлы на предмет наличия скрытых сообщений, используя различные методы стеганографии.

Stego Break, в свою очередь, является инструментом взлома стеганографической защиты. Он позволяет обнаруживать и уничтожать данные, скрытые по алгоритму наименее значащих битов (LSB), не повреждая при этом исходное содержимое файла.

Stego Suite предоставляет подробные отчёты о работе программы, включая методы, результаты и аномалии в цифровом медиафайле. Программа позволяет проводить детальный визуальный анализ изображения на уровне пикселей и находить артефакты, указывающие на скрытую информацию.

Приложение может внедрить информацию в медиафайл, предварительно зашифровав её и добавив в зашумлённые области. Однако большое количество искусственных фильтров на изображении может привести к ложным срабатываниям.

Программа работает с различными типами файлов, включая цифровые изображения и аудиофайлы, автоматически обнаруживает и удаляет скрытые сообщения, обеспечивая защиту конфиденциальной информации.

Stego Suite – это коммерческое программное обеспечение, которое распространяется только на платной основе.

Virtual Steganography Laboratory (VSL) [5] - бесплатное программное обеспечение для стеганографии и стеганализа изображений в форме графического инструмента для построения блоксхем. Оно позволяет комплексно использовать, тестировать и настраивать различные стеганографические методы и предоставляет простой графический интерфейс вместе с модульной архитектурой плагинов.

Приложение анализирует изображения на наличие скрытой информации и создаёт отчёт о результатах исследования. Оно также может скрывать сообщения в изображениях и работает с разными форматами.

Virtual Steganography Laboratory можно бесплатно скачать.

StegExpose [6] - инструмент для стеганографического анализа, предназначенный для обнаружения скрытой информации в изображениях, используя метод наименьшего значащего бита (LSB). Он разработан для работы с изображениями в формате без потерь, такими как PNG и BMP.

Основные характеристики StegExpose:

1. обнаружение LSB-стеганографии: StegExpose специализируется на обнаружении скрытой информации, внедрённой методом LSB;
2. анализ изображений без потерь: поддерживает анализ изображений в форматах без потерь, таких как PNG и BMP;
3. командная строка: имеет интерфейс командной строки, что облегчает автоматизацию процесса анализа;
4. составление отчётов: предоставляет возможность составления отчётов о результатах анализа;
5. настройка: позволяет настраивать параметры анализа для достижения наилучших результатов.

StegExpose не предназначен для глубокого изучения и анализа стеганографии, но может быть полезен для быстрой проверки наличия скрытой информации в изображениях.

StegoHunt MP [7] - передовое решение для обнаружения и анализа программ для сокрытия данных (стеганографии) и самих скрытых данных.

Основные функции StegoHunt MP включают:

1. обнаружение программ для стеганографии: StegoHunt MP способен обнаруживать наличие программ для сокрытия данных на компьютере или в сети. Это позволяет своевременно выявлять потенциальные угрозы и принимать меры по их устранению;
2. анализ файлов на предмет скрытых данных: система анализирует файлы на наличие скрытых данных, что особенно важно для цифровых носителей, таких как изображения, аудио и видеофайлы;

3. поддержка различных форматов файлов: StegoHunt MP поддерживает широкий спектр форматов файлов, включая изображения, аудио и видео, что делает его универсальным инструментом для анализа;

4. расширенная поддержка типов файлов: система способна анализировать файлы различных типов, включая изображения (JPEG, BMP, GIF, PNG), аудио (WAV, MP3) и видео (MP4, AVI, MOV);

5. улучшенный анализ изображений: StegoHunt MP предоставляет расширенные возможности анализа изображений, включая просмотр отдельных каналов (красный, синий, зелёный) и использование фильтров для более детального исследования;

6. конфигурируемые пороги обнаружения: пользователи могут настраивать пороги обнаружения для более точного определения наличия скрытых данных;

7. создание отчётов: StegoHunt MP генерирует подробные отчёты о результатах анализа, что упрощает процесс расследования и документирования;

8. интеграция с другими инструментами: система может быть интегрирована с другими инструментами для обеспечения комплексной защиты информации.

Приложение является важным инструментом для специалистов по информационной безопасности, позволяя им эффективно обнаруживать и анализировать скрытые данные. Программное обеспечение платное и не применяет машинное обучение.

Таблица 1.1 - Сравнение ПО для выявления информации в изображениях

Критерии для сравнения	StegDetect	Stegi Suite	VSL	StegExpose	StegoHunt MP
------------------------	------------	-------------	-----	------------	--------------

Способность обрабатывать большие объёмы данных	+	+	+	+	+
Автоматическое выявление скрытого содержимого	+	+	+	+	+
Поддержка различных форматов файлов	-	+	+	+	+
Бесплатное использование	+	-	+	+	-
Сигнатурный анализ	-	-	-	-	-
Статистический анализ	+	+	+	+	+
Машинное обучение	-	-	-	-	-

Данные приложения не выявляют сложные методы.

Анализ программных обеспечений показал, что используются только статистические методы для выявления стеганографических вставок.

Выводы по разделу 1

В этом разделе была описана предметная область работы, а именно, что такое стеганография, классификация методов внедрения информации, применение стеганографии, а также угрозы, реализуемые с использованием нее. Было проанализировано программное обеспечение, в котором

используются только статистические методы. В следующей главе будет обзор методов стегоанализа.

2 Обзор методов стегоанализа

2.1 Стегоанализ

Стегоанализ – это процесс поиска тайно передаваемой информации в исследуемом информационном объекте. Под скрытой информацией понимается информация, спрятанная с помощью различных методов стеганографии [8].

Стегоанализ может применяться как для обнаружения, так и для извлечения скрытой информации. В первом случае цель заключается в том, чтобы установить факт передачи скрытой информации, а во втором — восстановить эту информацию.

Для проведения стегоанализа используются различные методы и алгоритмы.

Классификация по принципу обработки информации [9]:

1. Сигнатурные методы предполагают визуальный (качественный) или синтаксический количественный анализ последовательности символов, составляющих контейнер, для выявления необычных элементов.

2. Статистические методы анализа контейнера используются для выявления расхождений между статистическими данными, полученными в ходе анализа (гистограммы, непараметрические статистики различных видов), и статистикой, характерной для «естественных» объектов, которые не были изменены в процессе внедрения сообщения.

3. Методы машинного обучения, которые строятся на создании классификаторов объектов для обнаружения внедрённых сообщений, используя представительные обучающие выборки, включающие заполненные и пустые контейнеры.

В свою очередь это класс делится на две крупные группы:

1. Традиционные «неглубокие» методы и алгоритмы машинного обучения, являющиеся относительно простыми (shallow methods);

2. Методы и алгоритмы, использующие глубокие нейронные сети (deep learning methods)

Сигнатурный методы:

Примеры сигнатурных методов стегоанализа включают:

Поиск отпечатков пальцев (fingerprints): Эти методы ищут специфические фрагменты кода или последовательности байтов, которые могут указывать на использование стеганографического программного обеспечения. Например, если известно, что конкретное стеганографическое приложение оставляет определенный шаблон в начале или конце файла, сигнатурный метод может искать этот шаблон для выявления возможного использования стеганографии.

Анализ заголовков файлов: Некоторые стеганографические методы изменяют заголовки файлов для скрытия информации. Сигнатурные методы могут анализировать заголовки файлов на предмет аномалий, которые могут указывать на наличие скрытого сообщения.

Проверка целостности файла: Сигнатурные методы могут проверять целостность файла, сравнивая его с известной контрольной суммой или хэшем. Любые изменения в файле, вызванные внедрением скрытого сообщения, могут привести к изменению контрольной суммы, что может быть обнаружено сигнатурным методом. Эти методы основаны на поиске специфических признаков или шаблонов, которые могут указывать на использование стеганографии.

2.2 Статистические методы:

2.2.1 Хи-квадрат [1]:

Этот метод направлен на обнаружение информации, скрытой методом наименьших значащих бит (НЗБ).

Суть метода заключается в анализе пар значений, которые кодируют интенсивности цветов и отличаются лишь на один наименьший значащий бит. Предполагается, что в незаполненном контейнере вероятность

одновременного появления обоих значений каждой пары мала, поэтому количество значений интенсивностей цветов, отличающихся на наименьший значащий бит, значительно разнится.

Теоретически ожидаемое распределение выбирается на основе средних арифметических значений частот всех пар. При скрывании информации методом НЗБ происходит лишь перераспределение частот внутри пары, поэтому сумма частот пары остаётся неизменной.

Наблюдаемая выборка сравнивается с теоретически ожидаемым распределением с помощью критерия Хи-квадрат. Если отклонения от теоретически ожидаемого распределения незначительны, это указывает на высокую вероятность встраивания информации.

Этот метод стегоанализа эффективен при применении к частям изображения, таким как блоки или условные строки матрицы пикселей. Это позволяет увидеть начало и конец последовательно встроенного сообщения. В целом можно использовать для разных методов, но в основном справляется с последовательной LSB вставкой.

2.2.2 RS-метод [1]

В RS-методе («регулярный-сингулярный») изображение разбивают на группы пикселей, к которым применяют флиппинг-процедуру. Затем, исходя из значения функции-дискриминанта до и после флиппинга, группы классифицируют как регулярные, сингулярные и неиспользуемые.

Принцип работы алгоритма заключается в том, что количество регулярных и сингулярных групп пикселей в исходном и обработанном изображениях должно быть примерно равным. Если это количество значительно изменяется, то изображение считается заполненным контейнером.

По шагам алгоритм работает следующим образом:

1. Изображение разделяется на группы из n пикселей $(g_1, g_2, \dots, g_n)$.
2. Описывается дискриминант-функция, которая сопоставляет

3. Каждой группе пикселей $G = (g_1, g_2, \dots, g_n)$

Мы можем определить функцию регулярности для группы пикселей $(g_1, g_2, \dots, g_n)$ следующим образом:

$$4. f(G) = f(g_1, g_2, \dots, g_n) = \sum_{i=1}^{n-1} |g_{i+1} - g_i| \quad (1)$$

Также мы определяем функцию флиппинга, которая обладает следующими свойствами:

$$1) \forall x \in P: F(F(x)) = x, P = \{0, 255\}$$

$$2) F_{\text{пр}}: 0 \leftrightarrow 1, 2 \leftrightarrow 3, \dots, 254 \leftrightarrow 255$$

$$3) F_{\text{обр}}: -1 \leftrightarrow 0, 1 \leftrightarrow 2, \dots, 255 \leftrightarrow 256$$

После выполнения операции переворота над группой происходит сравнение значений функции регулярности со значением до переворота. На основе этого сравнения группу относят к одному из классов: обычные (regular), необычные (singular), непригодные (unusable):

$$\text{Regular groups: } G \in R \leftrightarrow f(F(G)) > f(G)$$

$$\text{Singular groups: } G \in S \leftrightarrow f(F(G)) < f(G)$$

$$\text{Unusable groups: } G \in U \leftrightarrow f(F(G)) = f(G)$$

Флиппинг для каждой группы осуществляется дважды: с использованием прямой и инвертированной маски. Затем, после выполнения классификационных операций для всех групп, производится подсчёт ряда количественных характеристик:

- количество обычных групп для маски M : R_M
- количество необычных групп для маски M : S_M
- количество обычных групп для инверсной маски $-M$: R_{-M}
- количество необычных групп для инверсной маски $-M$: S_{-M}

Все описанные характеристики задаются как относительные величины, то есть в процентах от общего числа групп k . Таким образом, $R_M + S_M \leq 1$ и $R_{-M} + S_{-M} \leq 1$. Основная идея этого метода заключается в том, что в пустом

контейнере число групп одного класса для обычной и инверсной маски одинаково: $R_M \cong S_M$ и $R_{-M} \cong S_{-M}$

На рис. 1 представлен типичный вид так называемой RS-диаграммы – графиков значений R_M , S_M , R_{-M} и S_{-M} в зависимости от количества пикселей с инвертированными НЗБ в изображении.

На рисунке 1 график RS-диаграмма значений R_M , S_M , R_{-M} и S_{-M} в зависимости от количества пикселей с инвертированными LSB. На рис. 1. Ось x представляет собой процент пикселей с инвертированными LSB, ось y – относительное число регулярных и сингулярных групп с масками M и $-M$,

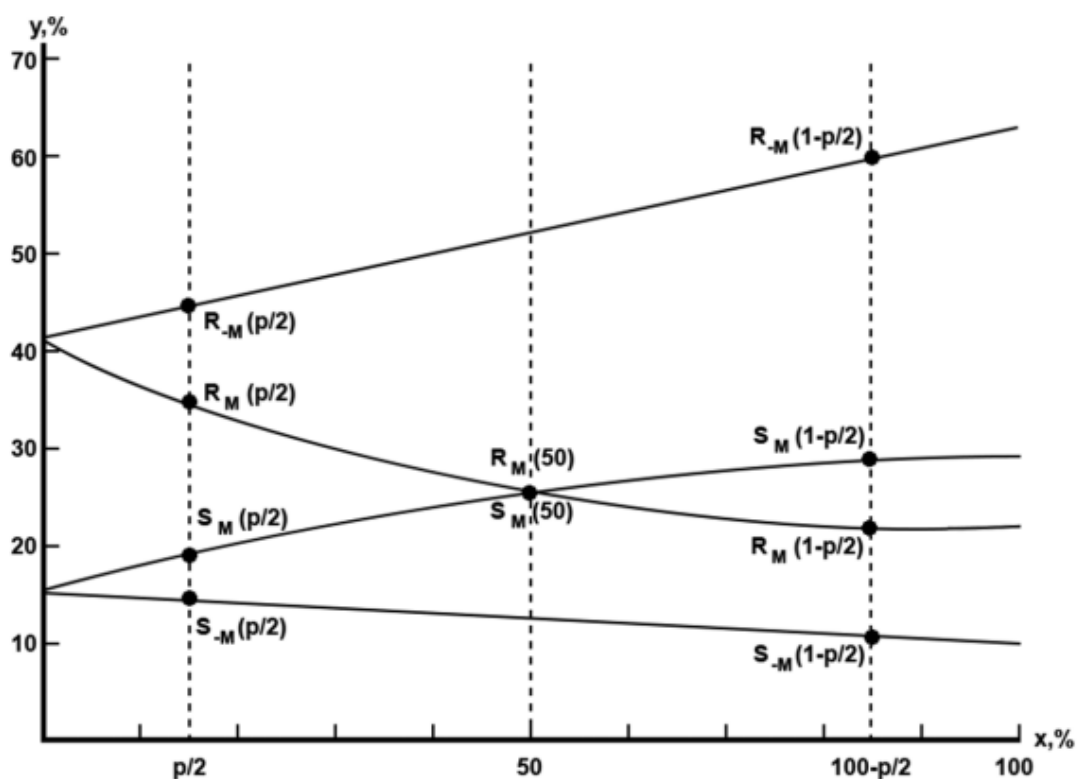


Рисунок 1 - RS-диаграмма

Исходя из данных о стандартном поведении графиков, отражающих количественные параметры групп, проводятся дополнительные расчёты, которые позволяют определить приблизительную длину скрытого сообщения.

Также можно посчитать длину с помощью формулы. Для оценки относительной длины сообщения p надо решить квадратное уравнение, которое получено на основе 11 уравнений с 11 неизвестными.

$$p = \frac{x}{x - \frac{1}{2}} \quad (2)$$

Особенность RS-метода заключается в анализе количественных характеристик небольших групп пикселей. Из-за этого метод не может определить область потенциального внедрения, но способен выявить скрывание, которое было произведено в случайные биты, а не последовательно.

Этот метод может обнаруживать как случайный, так и последовательный LSB.

2.3 Методы машинного обучения:

Машинное обучение — это раздел искусственного интеллекта, который даёт возможность компьютерам обучаться и подстраиваться под новые условия без непосредственного участия человека. Для этого компьютеру предоставляют большое количество данных и алгоритмы, которые помогают выявлять закономерности и делать прогнозы на основе новой информации [10].

Машинное обучение включает два основных подхода:

1. обучение с учителем, когда компьютер обучается на размеченных данных;
2. обучение без учителя, когда компьютер самостоятельно находит структуры в данных.

Перечислим часть методов:

1. Наивный байесовский классификатор - простой, но эффективный алгоритм. Он основан на теореме Байеса и предполагает независимость входных переменных. Этот метод хорошо подходит для решения сложных задач, таких как классификация спама или распознавание рукописных цифр.
2. Метод опорных векторов (SVM) - один из самых популярных и обсуждаемых алгоритмов машинного обучения. Он использует гиперплоскость для разделения точек данных на классы. Алгоритм ищет

оптимальную гиперплоскость, которая наилучшим образом разделяет точки данных.

3. Композиционные алгоритмы на основе бэггинга, такие как «случайный лес», представляют собой разновидность ансамблевого метода. Они создают множество моделей на различных подвыборках данных, а затем объединяют их прогнозы для повышения точности.

4. Композиционные алгоритмы на основе бустинга, например AdaBoost, также являются ансамблевыми методами. Они последовательно строят модели, каждая из которых пытается исправить ошибки предыдущей.

5. Метод k-ближайших соседей – это способ машинного обучения, который относит новый объект к классу на основе его сходства с объектами из тренировочного набора данных. Для этого алгоритм определяет k ближайших объектов в тренировочном наборе и присваивает новому объекту класс, который наиболее часто встречается среди этих k соседей.

6. Логистическая регрессия – это метод машинного обучения, применяемый для задач бинарной классификации. Он создаёт модель, которая оценивает вероятность принадлежности объекта к одному из двух классов на основании входных данных. Логистическая регрессия использует функцию логистического распределения для преобразования линейного предиктора в вероятность.

7. Случайный лес - совокупность методов машинного обучения, состоящая из множества деревьев решений. Каждое дерево решений обучается на случайно выбранной части данных и случайных подмножествах признаков. При классификации нового объекта случайный лес выбирает класс, который чаще всего встречается среди предсказаний отдельных деревьев.

2.4 Глубокие нейронные сети

Обзор глубокого обучения

В широком смысле глубокое обучение определяется как класс алгоритмов машинного обучения, которые используют многослойные

нейронные сети для извлечения более высокого уровня признаков из необработанных данных. Эти методы используют большие объемы обучающих данных для извлечения сложных функций. богатые данные для генеративных или классификационных задач.

Генеративно-состязательная сеть (GAN) - архитектура глубокого обучения, которая состоит из двух нейронных сетей: генератора и дискриминатора.

Генератор создаёт правдоподобные данные, которые служат негативными обучающими примерами для дискриминатора. В начале обучения генератор выдаёт заведомо ненастоящие данные, и дискриминатор быстро учится определять подделки, наказывая генератор за неправдоподобные результаты.

Со временем генератор учится создавать всё более реалистичные данные, а дискриминатор — лучше отличать подделки от настоящих данных.

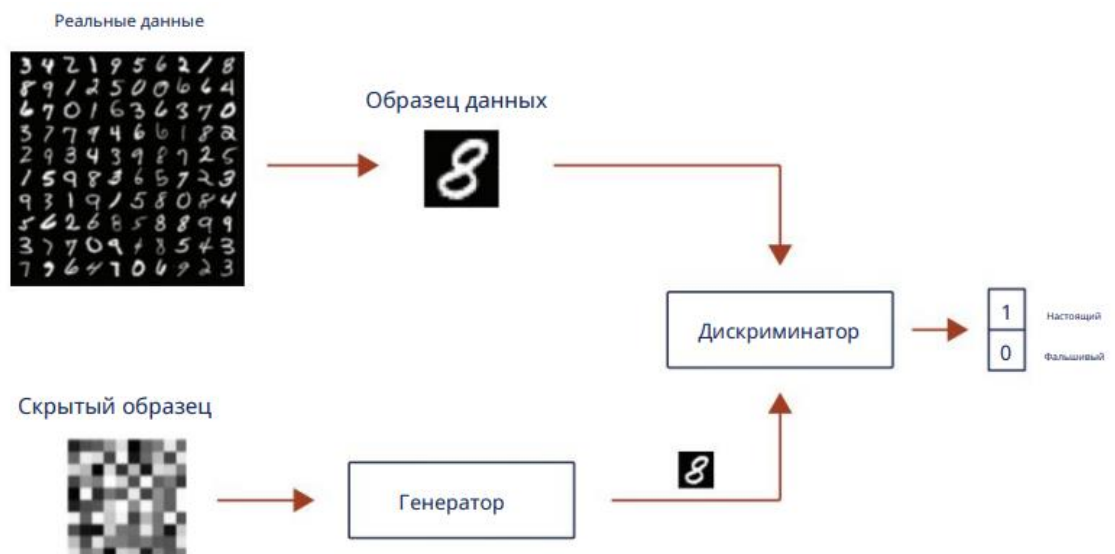


Рисунок 2 - Генеративно-состязательная сеть.

Эта состязательная установка позволяет генератору эффективно моделировать целевые распределения.

Свёрточные нейронные сети (СНС) эффективно используются для выявления пространственных закономерностей в цифровых изображениях.

Благодаря особенностям построения СНС, удаётся сократить число параметров и повысить точность определения признаков.

В общем случае СНС состоят из следующих базовых блоков:

- Свёрточные слои;
- Слои подвыборки (пулинга);
- Полносвязные слои.

Первый свёрточный слой обычно распознаёт признаки низкого уровня, а последующие слои объединяют их, находя признаки более высокого уровня. Исследования показывают, что обучение свёрточной нейронной сети на большом количестве цифровых изображений позволяет оптимально настроить веса свёрточных слоёв. Это нужно для преобразования изображений в вектор признаков, который может быть обработан вычислительной системой. Такой подход повышает эффективность распознавания при минимальной вычислительной сложности.

Кроме того, свёрточные нейронные сети могут находить определённые характеристики не только в целом цифровом изображении, но и в его отдельных частях. Это становится возможным благодаря сегментации изображения во время прохождения через ядро свёрточной нейронной сети.

Задача свёрточного слоя свёрточной нейронной сети (СНС) — выполнить двумерную свёртку, то есть операцию, которая уменьшает размер матрицы.

На рисунке 3 ядро свёртки — это матрица, состоящая из коэффициентов, которые называют весами. По сути, это фильтр. Когда ядро перемещается по двумерному изображению, которое тоже является матрицей, происходит следующее: веса умножаются на значения пикселей, над которыми находится ядро, а потом складываются. В результате получается значение нового элемента (пикселя) следующего слоя.

Таким образом, на выходе свёрточного слоя появляется новая матрица (изображение) меньшего размера.

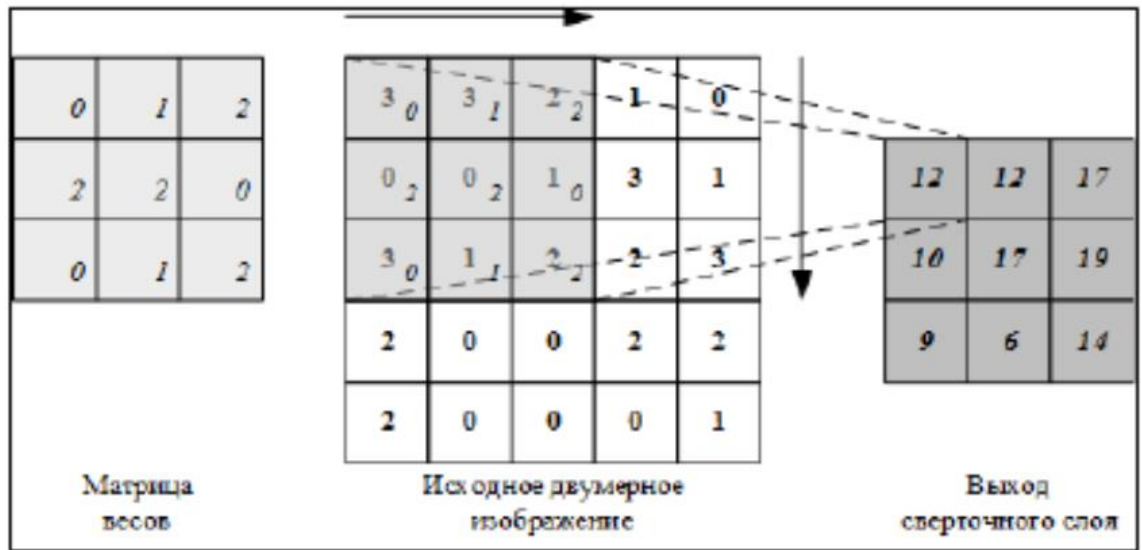


Рисунок 3 - Процесс двумерной свертки черно-белого цифрового изображения.

Как и свёрточный, слой подвыборки (пулинговый) нужен для уменьшения размера изображения и снижения количества необходимых вычислительных операций. Существует несколько видов пулинга: максимальный (Max Pooling), средний (Average Pooling) и пулинг суммы (Sum Pooling).

Первый метод используется для нахождения максимального значения в части изображения, которую покрывает ядро. Второй метод применяется для вычисления среднего значения среди всех элементов выбранной области. Третий метод служит для определения их суммы (рис. 4).

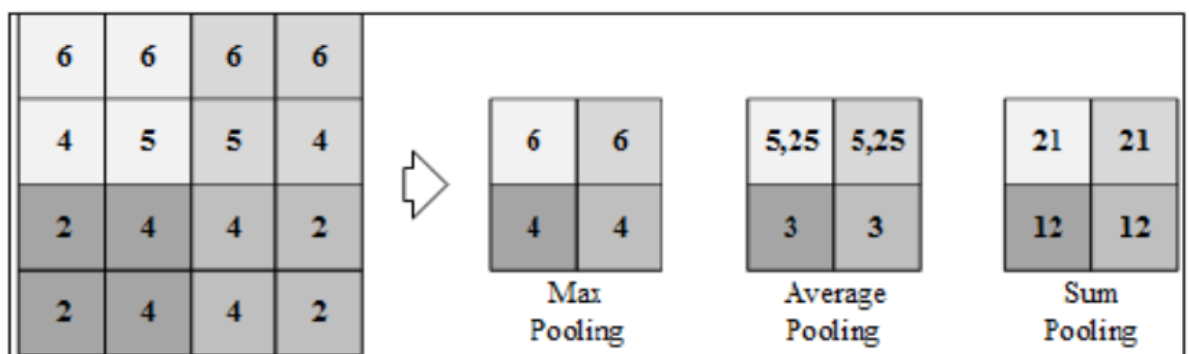


Рисунок 4 - Процесс обработки изображения своим подвыборки.

Полносвязный слой (рис. 5) моделирует нелинейную функцию, которая используется для классификации. Предыдущие слои нужны для предварительной обработки цифрового изображения.

Несколько слоёв обученной свёрточной нейронной сети позволяют находить закономерности во входных данных и анализировать их, чтобы получить решение задачи. После обучения свёрточной нейронной сети можно будет определить, к какому классу относится изображение, что важно для распознавания стегоконтейнеров.

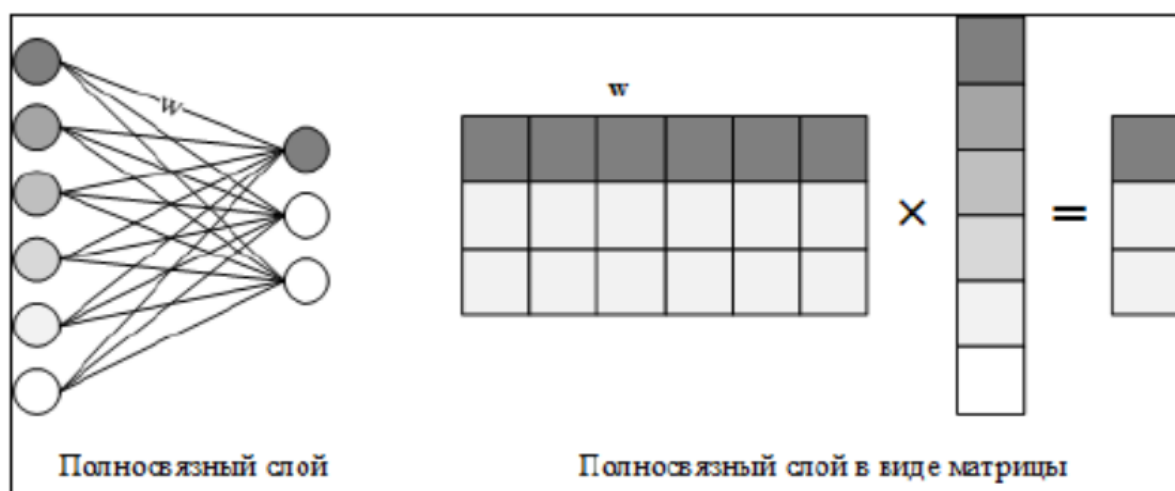


Рисунок 5 - Представление полносвязного слоя нейронной сети.

При проектировании сверточных сетей можно поступить так:

1. Использовать предобученную нейронную сеть.
2. Спроектировать с нуля.

Использование предобученной нейронной сети поможет ускорить процесс обучения. К достоинствам предложенного способа обнаружения стеговложений можно отнести достаточную точность и простоту реализации. Из недостатков стоит отметить необходимость решения задачи формирования представительной выборки цифровых изображений, используемой на этапе обучения нейронной сети.

При проектировании своей сети с нуля, придется определить архитектуру сети, выбрать количество слоев, типы слоев, функции активации и другие параметры. Этот процесс может быть более трудоемким, но он дает возможность полностью адаптировать сеть под нужды.

Методы глубокого обучения.

Стегоанализаторы глубокого обучения во многом зависят от своего набора обучающих данных. Было показано, что они являются наиболее эффективными инструментами стегоанализа по сравнению со статистическими стегоанализаторами.

Несмотря на то, что стегоанализаторы глубокого обучения были очень эффективны при стеганографическом обнаружении, они остаются относительно ненадежными. Фактически, недавние статьи показали, что стегоанализаторы глубокого обучения имеют большое количество видов отказов. Понимание и решение этих видов отказов остаётся одним из самых сложных препятствий, с которыми сталкиваются стегоанализаторы глубокого обучения.

2.5 Сравнение методов стегоанализа

Сигнатурный анализ:

Методы, основанные на поиске сигнатур, легко автоматизировать, поэтому они эффективны при обработке большого количества контейнеров без участия человека.

Методы стеганографии позволяют скрыть секретную информацию и изменить цифровые носители таким образом, чтобы изменения были незаметны человеческому глазу. Стеганография изменяет свойства носителя за счет вставки битов сообщения в форме деградации или повторяющихся шаблонов, которые действуют как подписи, передающие существование встроенного сообщения.

Существуют инструменты, позволяющие удалить стеганографическое содержимое из любого изображения. Это делается двумя способами: разделением и повторной выборкой.

Чтобы обнаружить скрытое сообщение в подозрительном изображении, нужно искать повторяющиеся шаблоны, которые являются сигнатурами инструментов стеганографии. Эти методы анализируют таблицы палитр в изображениях GIF и другие аномалии, созданные стандартными инструментами стеганографии.

Однако их работа базируется на знании алгоритма работы стеганодетектора, который можно определить лишь путём дизассемблирования. Недостатком сигнатурного подхода является невозможность выявления новых алгоритмов, информации о которых (и соответствующих сигнатур) ещё нет в базе данных используемого стеганоаналитического ПО.

Статистический анализ:

Этот метод основан на анализе процесса внедрения и определении определённых характеристик изображения. Он требует детального понимания процесса внедрения, то есть необходимо построить математическую модель «естественного» или «заполненного» контейнера. Создание точной модели «естественного» изображения или аудиосигнала является крайне сложной задачей. Иногда проще создать «заполненный» контейнер. Это затрудняет разработку эффективных статистических методов.

Точность результатов снижается, если метод применяется для обнаружения или извлечения скрытых сообщений из данных, которые были модифицированы с использованием определённой стеганографической техники.

Поскольку этот метод предполагает отсутствие знаний о процессе внедрения, его также называют слепым методом.

Для извлечения скрытого сообщения из стегоизображений, созданных с помощью техник LSB-встраивания, LSB-сопоставления, расширения спектра,

сжатия JPEG и других методов преобразования, применяются специальные статистические инструменты стегоанализа.

«Неглубокие» методы машинного обучения:

Эти методы включают в себя широкий спектр алгоритмов, таких как линейная регрессия, деревья решений, метод опорных векторов и другие. Они обладают рядом достоинств, включая способность выявлять закономерности даже в сложных данных и обрабатывать нелинейные взаимосвязи. Однако они также имеют некоторые недостатки, такие как необходимость вручную строить вектор признаков и высокая стоимость процесса обучения. Кроме того, эти методы ограничены данными, на которых они были обучены, то есть датасетом.

Методы, использующие глубокие нейронные сети:

Глубокое обучение — это подмножество машинного обучения, которое использует глубокие нейронные сети для решения сложных задач. Глубокие нейронные сети состоят из нескольких слоёв искусственных нейронов, которые могут обучаться распознаванию сложных паттернов в данных.

Достоинством глубоких нейронных сетей является их способность выявлять закономерности даже в сложных данных, обрабатывать нелинейные взаимосвязи и работать с большими объёмами данных. Кроме того, нейронные сети могут самостоятельно находить признаки в данных, что упрощает процесс обучения. Также существует возможность использовать предобученные сети, что позволяет ускорить процесс обучения для новых задач.

Однако глубокое обучение также имеет некоторые недостатки, такие как высокая стоимость процесса обучения и ограниченная применимость к данным, на которых нейронная сеть была обучена.

В целом, выбор метода машинного обучения зависит от конкретной задачи и доступных ресурсов.

Сравнение методов:

Сигнатурные:

Достоинства

- Обоснованность решений, так как есть в базе данных

Недостатки

- Невозможность выявления новых алгоритмов, информации о которых (и соответствующих сигнатур) еще нет в базе данных.

Статистические:

Достоинства

- Эффективность против простых методов стеганографии
- Существуют готовые методы (Xi-квадрат, Rs-метод)

Недостатки

- Необходимость построить математическую модель «естественного» либо «заполненного» контейнера для нового метода стегоанализа.
- Ложность срабатывая после сжатия изображений

Методы машинного обучения

«Неглубокие» методы и алгоритмы машинного обучения.

Достоинства

- Обработка нелинейных взаимосвязей
- Обработка больших и сложных данных

Недостатки

- Необходимость вручную строить вектор признаков
- Высокая стоимость процесса обучения
- Ограничено данными, на которых оно обучено, то есть датасетом

Методы и алгоритмы, использующие глубокие нейронные сети (deep learning methods)

Достоинства

- Обработка нелинейных взаимосвязей
- Обработка больших и сложных данных
- Нейронная сеть сама находит признаки

- Возможность использовать предобученные сети

Недостатки

- Ограничено данными, на которых нейронная сеть обучена, то есть датасетом
- Высокая стоимость процесса обучения

На основе анализа было принято решение использовать статический метод, а именно Rs-метод, для выявления LSB-метода (Least Significant Bit), как последовательного, так и рандомного. Этот выбор обусловлен высокой эффективностью Rs-метода в обнаружении скрытых сообщений, встроенных в цифровые контейнеры с использованием LSB-метода.

Кроме того, для анализа сложных методов стеганографии, где применение традиционных методов может быть ограничено, решено использовать сверточную нейронную сеть. Преимущество сверточных нейронных сетей заключается в их способности эффективно обрабатывать изображения, что делает их идеальным выбором для анализа сложных стегановложений.

Важным аспектом использования сверточных нейронных сетей является возможность применения предобученных моделей. Это позволяет значительно сократить время и ресурсы, необходимые для обучения сети с нуля, и обеспечивает высокую точность обнаружения стегановложений.

Выводы по разделу 2

Во второй главе было описано что такое стегоанализ, классификация методов. Проведено сравнение методов стегоанализа, что позволяет перейти к разработке системы, в основе которой будут методы Rs и сверточная нейронная сеть.

3 Разработка системы для выявления вставок

3.1 Архитектура системы

Система стегоанализа – это специальное программное обеспечение, направленных на обнаружение скрытых сообщений или данных, внедренных в цифровые объекты, такие как изображения, аудиофайлы или сетевые пакеты. Такие системы обычно применяются в целях обеспечения безопасности информации и контроля за её передачей, а также для борьбы с преступной деятельностью, связанной с использованием стеганографии.

Система стегоанализа может быть рассмотрена как фильтр изображений, предназначенный для обнаружения и анализа скрытой информации в цифровых изображениях. Этот процесс включает в себя применение различных техник анализа, таких как статистический анализ, исследование изменений в структуре файла, анализ цифровых артефактов и другие методы, чтобы выявить наличие скрытой информации.

В контексте фильтрации изображений, система стегоанализа действует как инструмент для проверки изображений на наличие скрытых сообщений, встроенных методами стеганографии. Это позволяет обеспечить безопасность информации и контроль за её передачей, предотвращая несанкционированное использование скрытых каналов передачи данных.

В данной дипломной работе предлагается использовать два метода для выявления внедрений в изображениях. Rs-метод подходит для обнаружения LSB-методов, а сверточная нейронная сеть эффективна при выявлении сложных методов внедрения.

Этапы стегоанализа изображений в системе:

1. Подача изображения на вход системы. Система получает изображение, которое необходимо проверить на наличие скрытого внедрения.

2. Анализ изображения RS-методом. Применяется к изображению для выявления признаков внедрения, основанного на замене наименее значимых бит (LSB).

3. Выдача результата RS-методом. Если RS-метод обнаруживает признаки внедрения, система сообщает о наличии внедрения. В противном случае система переходит к следующему этапу.

4. Анализ изображения сверточной нейронной сетью. Сверточная нейронная сеть (СНС) анализирует изображение для выявления признаков более сложных методов внедрения.

5. Выдача результата СНС. Если СНС обнаруживает признаки внедрения, система сообщает о наличии внедрения. В противном случае система считает, что внедрение отсутствует.

6. Блокировка изображения. Если хотя бы один из методов (RS или СНС) обнаружил признаки внедрения, изображение блокируется. Это означает, что оно не может быть использовано или передано дальше без дополнительной проверки.

3.2 Модель нейронной сети

Сейчас есть много моделей свёрточных нейронных сетей (CNN), у каждой из которых своё уникальное строение. Способность этих моделей точно классифицировать изображения говорит об их эффективности, и одним из основных критериев этой точности служит их производительность на наборе данных ImageNet [11].

Таблица 3.1 - Некоторые модели для классификации изображений

Модель	Точность, 1%*	Точность, 5%**
ResNet-34	73.314	91.420

ResNet-50	76.130	92.862
AlexNet	56.522	79.066
VGG-19	72.376	90.876
GoogleNet	69.778	89.530

* Доля случаев, когда класс, предсказанный с наибольшей точностью, соответствует реальному классу.

** Доля случаев, когда класс с самой высокой точностью совпадает с любым из пяти классов с самым высоким процентом предсказания.

Таблица 3.2 - Схематичное представление модели ResNet-50

Наименование слоя	Размер выходного изображения	Слои
Свёрточный 1	112x112	7×7 , шаг 2
Свёрточный 2	56x56	3×3 объединение с шагом 2
		$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
Свёрточный 3	28x28	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$
Свёрточный 4	14x14	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$
Свёрточный 5	7x7	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$

Средний слой объединения	1x1	Среднее объединение, полносвязный слой размером 1000 элементов с функцией активации softmax
Число параметров		$3.8 * 10^9$

У модели есть ряд преимуществ, хотя она и не является самой продвинутой на сегодняшний день. Во-первых, она показывает хорошую эффективность. Во-вторых, её структура не слишком сложна, что ускоряет обучение и упрощает интерпретацию результатов. Это особенно ценно, поскольку ресурсы для вычислений ограничены.

3.3 Датасет

Стегоанализ с использованием сверточных нейронных сетей требует наличия специализированного датасета для обучения и валидации модели. Этот датасет должен включать в себя два типа изображений: содержащие стегоинформацию и чистые, без нее.

Для создания датасета мы взяли за основу набор данных Image Dataset с сервиса Kaggle [12], который содержит более 82 тысяч изображений различных объектов [13]. Изображения в наборе имеют разный размер, для применения в глубоком обучении их преобразовали до размера 256x256 пикселей.

Использование больших изображений увеличило бы время обучения сети, а маленькие изображения редко применяются в стеганографии из-за ограниченного объема контейнера. К тому же ограничены ресурсы компьютера.

Из исходного набора данных, состоящего из 82 тысяч изображений, было выбрано 9000, поскольку модель уже была предварительно обучена и требовалось только её дообучения под наши нужды.

Информация была внедрена в 4500 изображений с использованием метода стеганографии Koch в RGB компоненту, а именно в синию. Процент заполнения контейнеров составил от 10 до 90, то есть на каждые 10% приходилось по 500 картинок. Koch с эпсилон 40. Была подобрана на основе графиков.

С помощью метрики PSNR (Peak Signal-to-Noise Ratio) получим количественную оценку изменения качества изображения после внедрения. Результаты представлены в децибелах (dB) для единственно измененного (синего) цветового канала изображения-контейнера, заполненного на 75%, при разных значениях эпсилона:

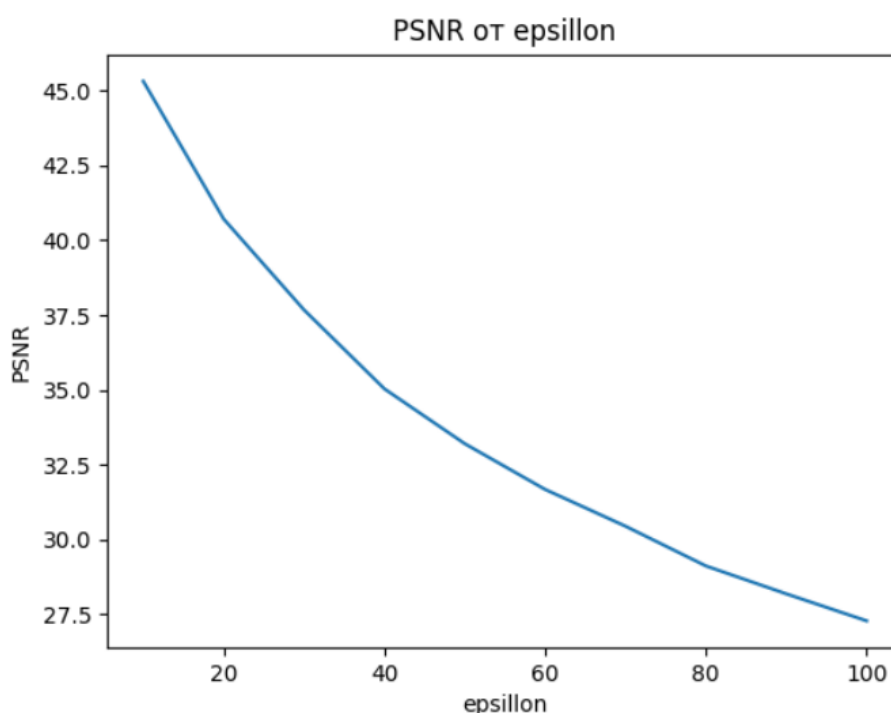


Рисунок 6. Зависимость PSNR от порогового значения эпсилона.

Также была проверена стойкость метода к сжатию при конвертации исходного bmp-изображения в jpeg (размер изображения уменьшился в 10 раз) и обратно. Результаты представлены для изображения-контейнера, заполненного на 75%, при разных значениях E:

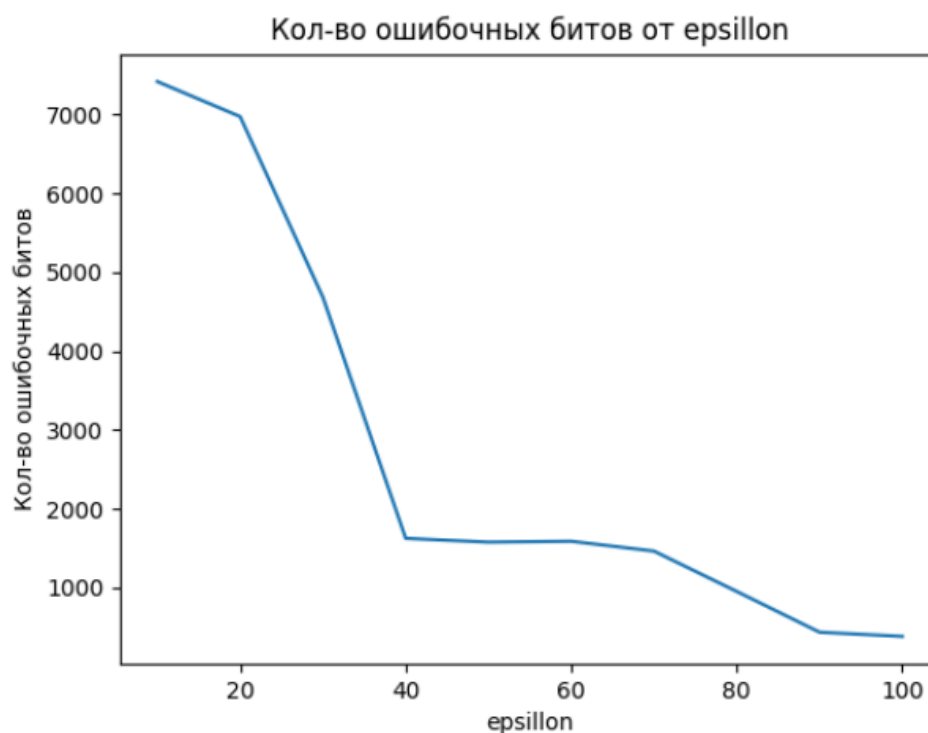


Рисунок 7. Зависимость количества ошибочных битов от порогового значения эпсилоне.

По этому графику видно, что при $E=40$ и выше количество ошибочных битов уже не так сильно меняется и составляет меньше 10% от общего числа. Поэтому можно сделать вывод о том, что эффективно использовать именно это значение, так как при нем же обеспечивается минимальная заметность при визуальном осмотре ($PSNR = 35$ dB).

Чтобы модель могла работать с набором данных, необходимо сохранить его в файле формата .csv. Этот текстовый формат используется для представления табличных данных.

1	file,class	8975	Koch_Zhao975.bmp,1
2	Empty0.bmp,0	8976	Koch_Zhao976.bmp,1
3	Empty1.bmp,0	8977	Koch_Zhao977.bmp,1
4	Empty10.bmp,0	8978	Koch_Zhao978.bmp,1
5	Empty100.bmp,0	8979	Koch_Zhao979.bmp,1
6	Empty1000.bmp,0	8980	Koch_Zhao980.bmp,1
7	Empty1001.bmp,0	8981	Koch_Zhao981.bmp,1
8	Empty1002.bmp,0	8982	Koch_Zhao982.bmp,1
9	Empty1003.bmp,0	8983	Koch_Zhao983.bmp,1
10	Empty1004.bmp,0	8984	Koch_Zhao984.bmp,1
11	Empty1005.bmp,0	8985	Koch_Zhao985.bmp,1
12	Empty1006.bmp,0	8986	Koch_Zhao986.bmp,1
13	Empty1007.bmp,0	8987	Koch_Zhao987.bmp,1
14	Empty1008.bmp,0	8988	Koch_Zhao988.bmp,1
15	Empty1009.bmp,0	8989	Koch_Zhao989.bmp,1
16	Empty101.bmp,0	8990	Koch_Zhao990.bmp,1
17	Empty1010.bmp,0	8991	Koch_Zhao991.bmp,1
18	Empty1011.bmp,0	8992	Koch_Zhao992.bmp,1
19	Empty1012.bmp,0	8993	Koch_Zhao993.bmp,1
20	Empty1013.bmp,0	8994	Koch_Zhao994.bmp,1
21	Empty1014.bmp,0	8995	Koch_Zhao995.bmp,1
22	Empty1015.bmp,0	8996	Koch_Zhao996.bmp,1
23	Empty1016.bmp,0	8997	Koch_Zhao997.bmp,1
24	Empty1017.bmp,0	8998	Koch_Zhao998.bmp,1
25	Empty1018.bmp,0	8999	Koch_Zhao999.bmp,1
26	Empty1019.bmp,0	9000	Koch_Zhao999.bmp,1
27	Empty102.bmp,0	9001	Koch_Zhao999.bmp,1

Рисунок 8. Размеченный датасет.

Для каждого изображения создаётся отдельная строка таблицы с метриками. Такая строка содержит метку и указывает, есть ли в контейнере стегоинформация. На рисунке 8 показано как выглядит датасет. Исходный код для создания датасета, приведён в «Приложении А».

Выводы по разделу 3

В 3 разделе была разработана система выявления стеганографических вставок в изображениях, работающая по принципу: если один из методов обнаружит вставку, считается что она существует. В качестве предобученной модели выбрана ResNet-50, для которой был создан датасет для выявления Koch метода.

Третий раздел формирует основу для реализации системы выявления вставок.

4 Реализация и тестирование системы выявления вставок

4.1 Описание среды разработки, обучение модели машинного обучения.

Обучение нейронной сети проходило на платформе Kaggle, предназначенной для состязаний по анализу данных и общения специалистов по работе с данными и машинному обучению, так как ресурсы компьютера не позволяют быстро обучить модель.

Здесь можно выкладывать датасеты, изучать и создавать модели, общаться с коллегами, проводить и участвовать в соревнованиях по анализу данных.

Сервис предлагает облачные инструменты для обработки данных и машинного обучения, а также образовательные ресурсы. Для высокопроизводительных вычислений Kaggle предоставляет GPU T4, GPU P100, TPU v3-8.

Kaggle Notebook – это облачная среда в Kaggle для специалистов по работе с данными и машинному обучению, которая поддерживает языки программирования Python и R и предоставляет инструменты и библиотеки для анализа данных и машинного обучения.

В качестве языка был выбран Python [14], поскольку существуют предобученные нейронные сети во фреймворке PyTorch [15]. Это библиотека глубокого обучения, в которой уже определены архитектуры моделей для обучения, необходимые инструменты для обучения и некоторые базовые датасеты.

В собственном датасете изображения делятся на два класса: наличие внедрения и его отсутствие. Детали формирования датасета обсуждались в разделе «Датасет».

С помощью инструментов PyTorch были загружены заранее подготовленные данные и архитектура ResNet50, в которой классификатор был изменён.

Классификатор состоит из пяти полносвязных слоёв, последним из которых является пятый выходной. Он содержит два нейрона, соответствующие двум классам распознавания.

Остальные четыре слоя содержат 512, 256, 128 и 64 нейрона соответственно, в порядке следования номеров слоёв. В этих слоях используется функция активации ReLu.

Модель обучалась в течение 10 эпох с использованием оптимизатора Adam, который представляет собой метод стохастического градиентного спуска, основанный на адаптивной оценке моментов первого и второго порядка [16]. Функция потерь использовалась кросс-энтропией.

Исходный код, реализующий процесс обучения, приведён в «Приложении Б».

4.2 Нужные метрики, анализ эффективности обучения выбранной модели машинного обучения.

Для оценки эффективности обучения используются различные метрики, каждая из которых предназначена для конкретного типа задачи и помогает выявить сильные и слабые стороны модели.

Точность (ассигасу) — это показатель, который говорит о том, насколько точно модель определяет класс объекта или события. Этот параметр служит общей оценкой эффективности модели и позволяет оценить, насколько хорошо модель может правильно классифицировать объекты или события. Чем выше точность, тем лучше модель выполняет задачу классификации.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \quad (3)$$

Функция потерь (Loss) определяет разницу между значениями, предсказанными моделью, и истинными значениями целевой переменной. На основании этих значений происходит оптимизация параметров модели в процессе обучения.

Она сравнивает реальные результаты, которые выдаёт модель в процессе обучения, с ожидаемыми. Чем меньше значение потерь, тем лучше модель справляется со своей задачей. Это значит, что обучение проходит успешно и параметры модели не нужно сильно изменять. Если же потери большие, то параметры модели необходимо скорректировать. Значение потерь рассчитывается с помощью функции потерь, которая уникальна для каждой модели.

Точность (precision) – это показатель, который отражает, насколько точно модель предсказывает положительный класс.

Он демонстрирует способность модели избегать ошибок, то есть правильно определять положительные примеры среди всех предсказанных положительных случаев. Чем выше точность, тем меньше ошибок модель совершает при прогнозировании положительного класса.

$$\text{Precision} = \frac{TP}{TP+FP} \quad (4)$$

Полнота (recall) — это метрика, которая показывает, насколько хорошо модель может находить положительные примеры класса. Она демонстрирует способность модели обнаруживать нужные нам объекты или события среди всех возможных положительных случаев. Чем выше recall, тем лучше модель справляется с поиском положительных примеров.

$$\text{Recall} = \frac{TP}{TP+FN} \quad (5)$$

F-мера (F1-Score)– среднее гармоническое между точностью(precision) и полнотой (recall) двумя ключевыми показателями в машинном обучении. Если точность или полнота стремятся к нулю, F-мера тоже стремится к нулю. И наоборот, F-мера достигает максимума, когда оба показателя максимальны. Эта метрика сводит информацию о точности и полноте к одному числу, что упрощает сравнение эффективности разных моделей.

$$F1 - Score = \frac{2 * (Precision * Recall)}{Precision + Recall} \quad (6)$$

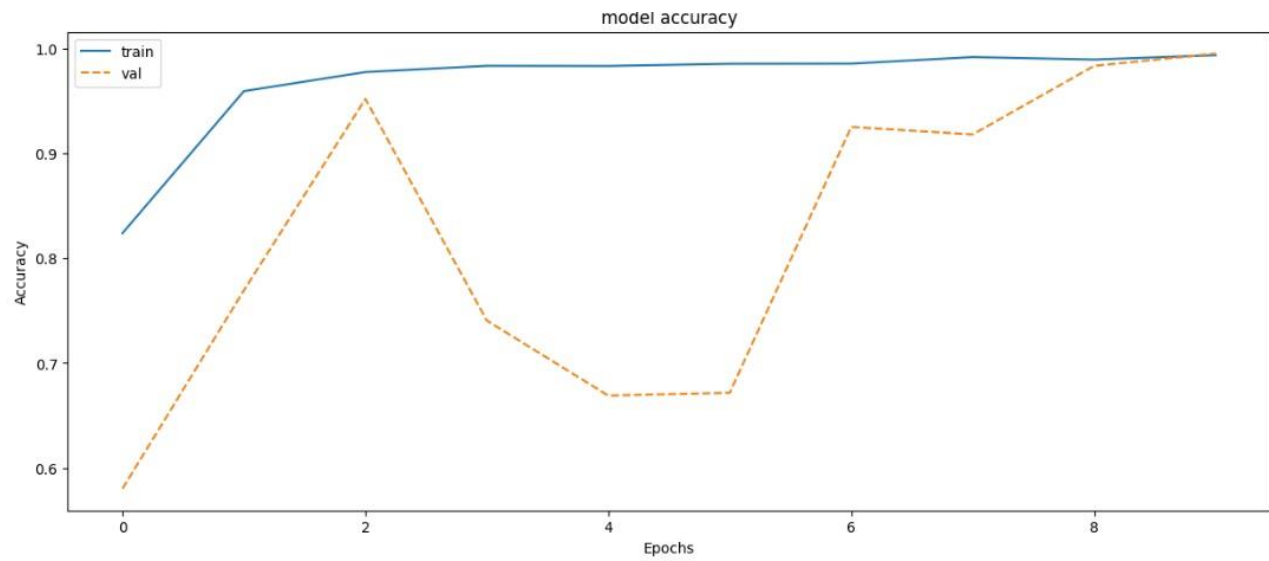


Рисунок 9 - График зависимости значений метрики accuracy от эпохи.

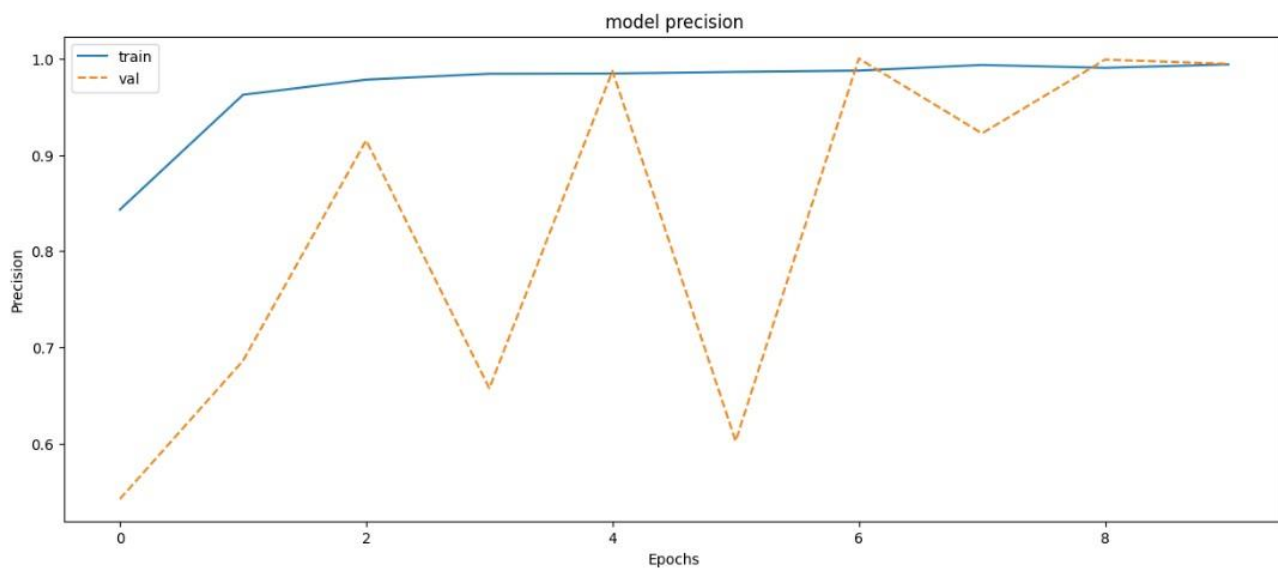


Рисунок 10 - График зависимости значений метрики precision от эпохи.

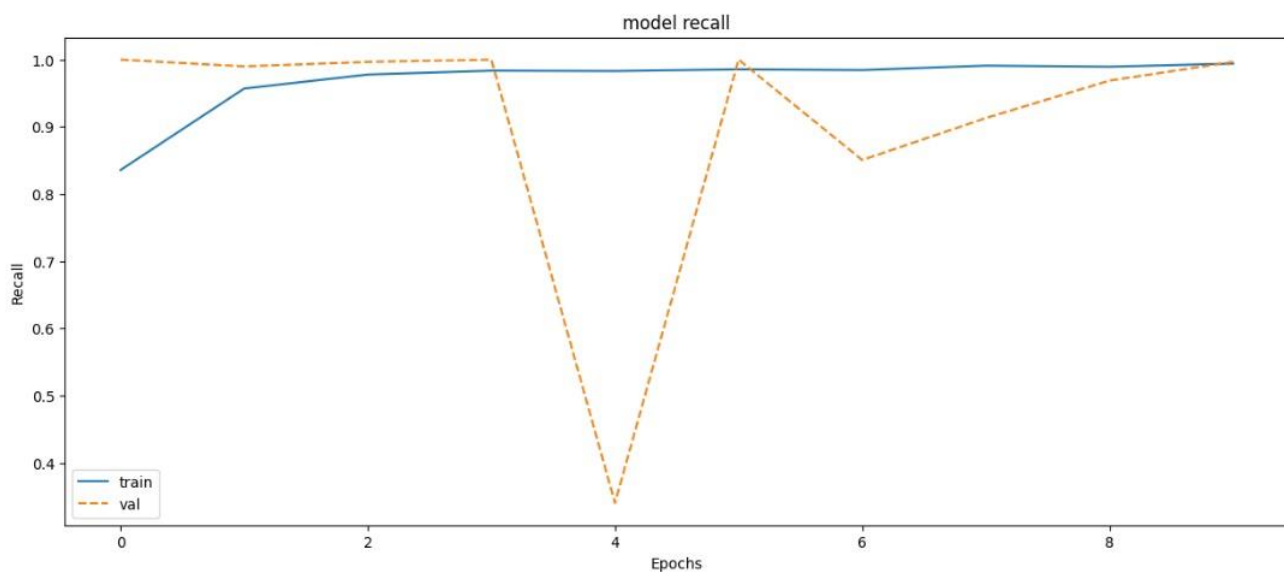


Рисунок 11 - График зависимости значений метрики recall от ЭПОХИ.

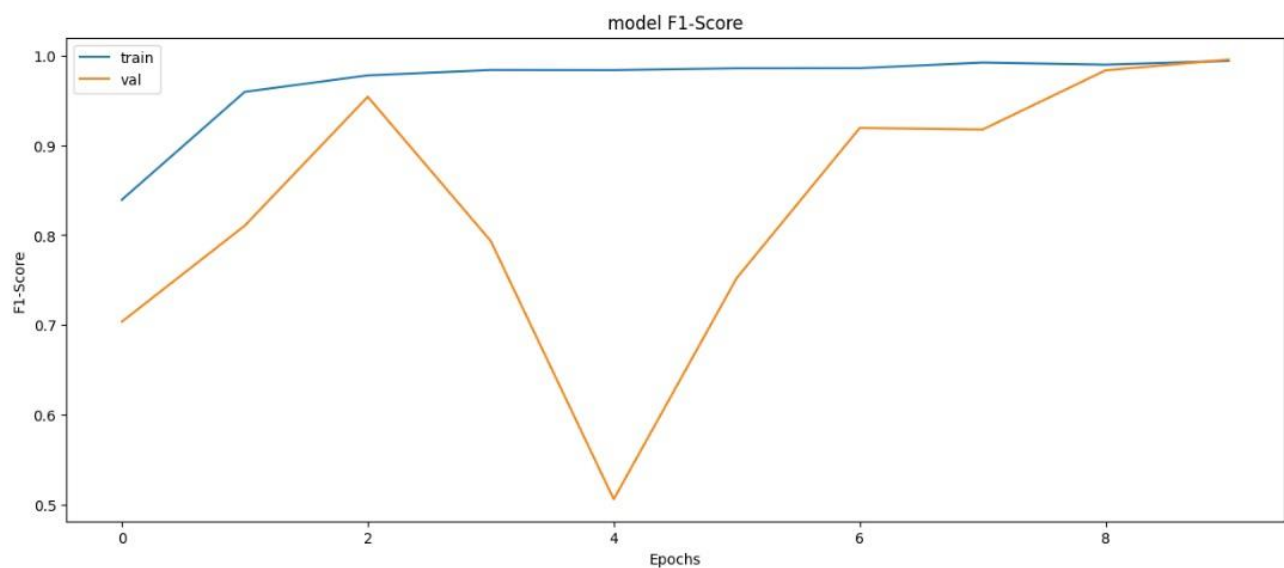


Рисунок 12 - График зависимости значений метрики F-Score от ЭПОХИ.

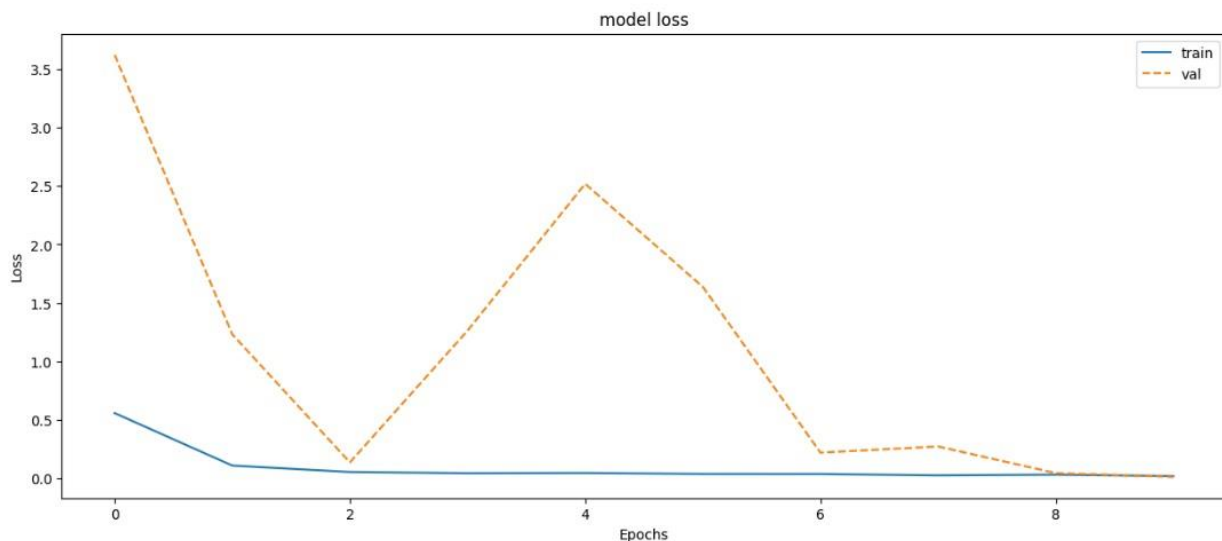


Рисунок 13 - График зависимости значений функции потерь loss от эпохи.

Исходя метрик: accuracy, precision, recall и F-Score, графики тренировочного и валидационного тестов в конце обучения совпали, а также близки к единице, следовательно, что модель обучилась отлично для выявления вставок методом Koch.

По завершению обучения, accuracy, precision, recall и F-Score показывают значения соответственно 0.994, 0.994, 0.994, 0.994 для тренировочного теста, а для валидационного 0.995, 0.994, 0.997, 0.996 это значит, что класс изображения, скорее всего, будет определён правильно.

4.3 Тестирование RS-метода и модели машинного обучения в совокупности

Для проверки системы использовались изображения, не входящие в датасет. Изображения были выбраны с заполненностью от 1% до 100% с шагом 10%. Всего было по 100 вставок в каждом шаге.

Исходный код, реализующий процесс тестирования системы, приведён в «Приложении В».

Таблица 4.1 - Анализ изображений “пустых”, LSB, Koch

Изображение	Количество изображений	Rs-сумма	CNN-сумма	Результат
Пустые	1000	84	0	916
LSB	1000	941	1	941
Koch	1000	80	989	995

На основе результатов в приведенных таблице 4.1, делается вывод, что нейронная сеть не видит LSB-вставки, но зато хорошо выявляется Koch. Rs-метод также выявил много LSB-вставок. Rs-метод определил 916 пустых картинок.

Таблица 4.3 - Анализ изображений с различным шумом

Тип шума	Количество изображений	Rs-сумма	CNN-сумма	Результат
Гауссовский (нормальный)	1000	976	369	977
Импульсный	1000	982	21	982
Пуассоновский	1000	699	369	769
Мультипликативный	1000	686	296	729

Система в целом реагирует на шумы, в основном из-за Rs-метода.

Таблица 4.4 - Анализ одноцветных изображений.

Монотонные цветные изображения	Количество изображений	Rs-сумма	CNN-сумма	Результат
Пустые	1000	0	14	986
LSB	1000	874	14	879
Koch	1000	213	919	934

Система выявляет в целом есть вставка или нет в одноцветных изображениях.

Таблица 4.5 - Анализ изображений с внедренной вставкой Koch в компоненту RGB

Изображение Koch	Количество изображений	Rs-сумма	CNN-сумма	Результат
Red	1000	81	461	518
Green	1000	69	368	424
Blue	1000	66	1000	1000

Анализ изображений с внедренной вставкой Koch в компоненту RGB

Система может выявлять в разных в компонентах RGB, однако лучше дообучить CNN выявлять компоненты Red и Green.

Таблица 4.6 - Анализ изображений с внедренной вставкой Benham в компоненту RGB и YCbCr.

Изображение Benham	Количество изображений	Rs-сумма	CNN-сумма	Результат
RGB (40)	1000	201	21	218
RGB (60)	1000	258	358	549
YCbCr (60)	1000	392	15	396

Система может выявлять вставки с помощью другого метода внедрения в области преобразований. Однако CNN плохо выявляет их в YCbCr, и только благодаря RS-методу система смогла определить вставку.

CNN обучалась на изображениях с внедрёнными вставками Koch в RGB, поэтому смогла выявить их в RGB при пороге 60, но не при 40. Это связано с тем, что, скорее всего, при пороге 60 признаки становятся похожими.

Следовательно, нейронную сеть необходимо научить выявлять разные методы внедрения в области преобразования, а также учитывать цветовые модели.

Выводы по разделу 4

В этом разделе мы дообучили ResNet-50 для выявления стеганографических вставок, в частности, Koch. По метрикам accuracy, precision, recall и F-Score, модель показала значения, близкие к единице по тестовому и валидационному тестам.

Также мы протестировали систему. Она хорошо выявляет обычные изображения и одноцветные, в которые внедрены вставки LSB и Koch, был сделан вывод, что система реагирует на шумы.

Нейронную сеть необходимо научить выявлять различные методы внедрения в области преобразования, а также учитывать цветовые модели.

Заключение

В данной дипломной работе разработана система выявления стеганографических вставок в изображениях, особенность данной идеи в том, чтобы использовать RS-метод для LSB-вставки, а сверточную нейронную сеть для Koch метода. В рамках данной дипломной работы были решены следующие задачи:

1. Исследованы способы применения стеганографии.
2. Проведен обзор существующих программ для выявления вставок.
3. Проведено сравнение методов стегоанализа.
4. Разработана архитектура системы, состоящая из двух методов, а именно RS-метод для LSB и СНС для сложных методов.
5. Была выбрана предобученная модель нейронной сети.
6. Разработан и создан датасет для обучения, валидации модели.
7. Проведен анализ эффективности системы.

Предложенная система позволяет выявлять LSB-вставки, а также Koch-вставки. Необходимо дообучить выявлять другие стего-вставки.

Главный принцип работы системы в том, чтобы простые методы выявлять посредством статистических, а сложные с помощью СНС.

Список используемых источников

- 1 В. А. Федосеев. Цифровые водяные знаки и стеганография [Текст]. – Самара: СГАУ, 2015 – 128 с.
- 2 Цифровые водяные знаки и стеганография: учебное пособие с заданиями для практических и лабораторных работ / В.А. Федосеев. – Электрон. текстовые и граф. дан. – Самара: СГАУ, 2015. – 128 с. – 1 эл. опт. диск (CD-ROM)
- 3 Stegdetect, URL: <https://www.provos.org/p/detection-with-stegdetect/> (дата обращения: 20.04.2024).
- 4 Stego Suite, URL: <https://stegosuite.org/about/> (дата обращения: 20.04.2024).
- 5 Virtual Steganography Laboratory, URL: <https://vsl.sourceforge.net/> (дата обращения: 20.04.2024).
- 6 StegExpose, URL: <https://github.com/b3dk7/StegExpose> (дата обращения: 20.04.2024).
- 7 StegoHunt MP, URL www.wetstonetech.com/products/stegohunt-steganography-detection/ (дата обращения: 20.04.2024).
- 8 Г. Ф. Конахович, А. Ю. Пузыренко. Компьютерная стеганография. Теория и практика [Текст]. – Киев: МК-Пресс, 2006 – 288 с.
- 9 П.В. Юренский. Методы статистического и нейросетевого стегоанализа скрытых каналов [Текст]. Журнал «Инновации в науке» № 1 (89), 2019 г
- 10 Г. Ф. Конахович, А. Ю. Пузыренко. Компьютерная стеганография. Теория и практика [Текст]. – Киев: МК-Пресс, 2006 – 288 с.
- 11 ImageNet, URL: <https://image-net.org/> (Дата обращения 22.04.2024)
- 12 Kaggle, URL: <https://www.kaggle.com/> (Дата обращения 15.04.2024).

- 13 Готовый набор данных с kaggle “Image dataset” URL: <https://www.kaggle.com/datasets/starktony45/image-dataset> (дата обращения: 18.04.2024)
- 14 PyTorch, URL: <https://pytorch.org/> (Дата обращения 15.03.2024).
- 15 Оптимизатор Adam, URL: <https://arxiv.org/abs/1412.6980> (дата обращения: 17.03.2023).
- 16 ImageNet Large Scale Visual Recognition Challenge, URL: <https://image-net.org/challenges/LSVRC/> (дата обращения: 25.04.2024)
- 17 А.И. Галушкин. Нейронные сети: основы теории. – М.: РиС, 2015.
- 18 С. Хайкин. Нейронные сети: полный курс. – М.: Диалектика, 2019.
- 19 М. Лутц. Изучаем Python. 5-е изд. – М.: Диалектика, 2019.
- 20 Р. Каллан. Нейронные сети: Краткий справочник. – М.: Вильямс, 2017.
- 21 П. Бэрри. Изучаем программирование на Python. – М.: Эксмо, 2016.

Программный код на языке Python для создания датасета

```
# ищем файлы в папке
# исходя из названия определяем класс
# получаем csv
import os
import pandas as pd
import numpy as np

# базовые классы
classes = ["Empty", "Koch_Zhao"]
files_class = []
# Указываем путь к директории
directory = "directory"
# Получаем список файлов
files_stega = os.listdir(directory)

# на основе на названия определяем класс
for i in range(len(files_stega)):
    for j in range(len(classes)):
        if classes[j] in files_stega[i]:
            files_class.append(j)
            break

print(len(files_class))
dataset={'file':files_stega,'class':files_class}
df = pd.DataFrame(dataset)
print(df)
df.to_csv('dataset.csv',index=False,header=True)
```

Программный код на языке Python для обучения нейронной сети

```
import pickle
```

```
import math
```

```
import numpy as np
```

```
import pandas as pd
```

```
from skimage import io
```

```
from matplotlib import colors, pyplot as plt
```

```
%matplotlib inline
```

```
from PIL import Image
```

```
from pathlib import Path
```

```
import torchvision
```

```
import torch
```

```
import torch.nn as nn
```

```
from torchvision import transforms, models
```

```
#from sklearn.preprocessing import LabelEncoder
```

```
#from sklearn.metrics import f1_score
```

```
from torch.utils.data import Dataset, DataLoader
```

```
from torch import optim
```

```
from tqdm import tqdm_notebook
```

```
import warnings
```

```
warnings.filterwarnings(action='ignore', category=DeprecationWarning)
```

```
import os
```

```
# importing Libraries
```

```
from sklearn.metrics import confusion_matrix, accuracy_score,  
precision_score, recall_score, f1_score, roc_curve, auc
```

```
import numpy
```

```
import gc
```

```
#параметры
```

```
#устанавливаем тренировочные гиперпараметры
```

```
num_epochs = 10 #эпохи
```

```
num_classes = 2#классы
```

```
batch_size = 100#батчи
```

```
batch_size_val = 100
```

```
learning_rate = 0.003 #время обучения
```

```
#создали датасеты
```

```
class SteganographyDataset(Dataset):
```

```
    def __init__(self, csv_file, root_dir, transform=None):
```

```
        self.annotations = pd.read_csv(csv_file)
```

```
        self.root_dir = root_dir
```

```
        self.transform = transform
```

```
    def __len__(self):
```

```
        return len(self.annotations)
```

```
    def __getitem__(self, index):
```

```
        img_path = os.path.join(self.root_dir, self.annotations.iloc[index, 0])
```

```
        image = Image.open(img_path)
```

```
        y_label = torch.tensor(int(self.annotations.iloc[index, 1]))
```

```
    if self.transform:
```

```
        image = self.transform(image)
```

```

        return (image, y_label)

#Путь выгрузки
#путь
ROOT_DIR = Path('/kaggle/input/data-set-verison-2/dataset v1/directory')
CSV_DIR = Path('/kaggle/input/data-set-verison-2/dataset v1/dataset.csv')
#Разбивка датасета
train_size=7200
test_size=1800

dataset=SteganographyDataset(csv_file=CSV_DIR,root_dir=ROOT_DIR,transform = transforms.ToTensor())#transforms.ToTensor()

# создаем загрузчики
train_set, test_set = torch.utils.data.random_split(dataset, [train_size, test_size])

train_loader = DataLoader(dataset=train_set, batch_size=batch_size, shuffle=True)

test_loader = DataLoader(dataset=test_set, batch_size=batch_size_val, shuffle=True)

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
DEVICE_CPU = torch.device("cpu")

#Обучение
# подготовка
# 1. Задать модель - nn.Module
#заменить классификатор для предобученной модели
# 1. Задать модель - nn.Module
#n_classes = len(np.unique(train_val_labels))
#model = models.efficientnet_b2(weights='DEFAULT')
#num_features = 1408
#model.classifier = nn.Sequential(
#
#    nn.Linear(num_features, 256),
#
#    nn.ReLU(),

```

```

#             nn.Linear(256, 23)
#)

model = torchvision.models.resnet50(weights="DEFAULT")

model.classifier = nn.Sequential(
    nn.Linear(512, 256),
    nn.ReLU(),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, num_classes)
)

# поместить модель и метрику на GPU
model.to(DEVICE)

#print("We will classify {} labels\n".format(num_classes))
print("We changed the last classifier layer with:\n", model.classifier)

# 2. Задать функцию потерь
# выбираем функцию потерь
#зависит от функции активации!!!!!!!!!!!!!!!!!!!!!!
loss_func = torch.nn.CrossEntropyLoss() #!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

# 3. Задать оптимизатор
# выбираем алгоритм оптимизации
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
# создать вспомогательные списки для данных
# н-р, лосс на каждой итераций

```



```
losses_train = []
# н-р, общий лосс
total_loss = []
# н-р, accuracy
acc_train = []
# н-р, accuracy
total_acc_train = []
# н-р, accuracy

# н-р, precision
prec_train = []
# н-р, precision
total_prec_train = []
# н-р, precision

# н-р, recall
recall_train = []
# н-р, recall
total_recall_train = []
# н-р, recall

# н-р, recall
f1_train = []
# н-р, recall
total_f1_train = []
# н-р, recall

acc_test = []
```

#----- ВАЛИДАЦИЯ -----

losses_val = []

total_loss_val = []

собираем acc

total_acc_val = []

собираем Precision

total_prec_val = []

собираем Recall

total_recall_val = []

собираем F1-Score

total_f1_val = []

#С метриками написанными вручную

Для каждой эпохи

for epoch in tqdm_notebook(range(num_epochs)):

#подсчет acc

num_correct = 0

num_samples = 0

#подсчет метрик

TP = 0

FP = 0

FN = 0

loss_train = 0

для каждой части датасета

for iterat, batch in tqdm_notebook(enumerate(train_loader)):

#print("iter"+str(iterat)+"\n")

ЭТАП ОБУЧЕНИЯ

```

# мы переводим модель в режим обучения
model.train()

#получаем текущий батч
X_batch, y_batch = batch #X_b - тензор картинки, y_b - класс
# ОБНУЛЯЕМ градиенты у оптимизатора
optimizer.zero_grad()

# Прямое распространение - пропускаем данные через модель
y_prediction = model(X_batch.to(DEVICE)) # полученное
предсказанное значение


# выравниваем выходы в одномерный тензор
# скорее всего в сверточной не надо,хз разберемся


# Вычисление функции потерь
# считаем лосс
loss = loss_func(y_prediction,y_batch.to(DEVICE))
# Обратное распространение
# делаем шаг в обратном направлении
loss.backward()
# делаем шаг оптимизатора
optimizer.step()
# собираем лоссы
#print("Тренировочный лосс \n")
#print(loss.detach().cpu().numpy().item())
losses_train.append(loss.detach().cpu().numpy().item())
loss_train = loss_train + loss.detach().cpu().numpy().item()
# перевод в нормальное значение


# print(y_batch.size())
# print("\n")

```

```

    #print(y_prediction.size())
    _, predictions = y_prediction.max(1)
    #собираем acc
    num_correct += (predictions == y_batch.to(DEVICE)).sum()
    num_samples += predictions.size(0)
    #подсчет параметров для метрик
    TP = TP + ((predictions.to(DEVICE) == 1) & (y_batch.to(DEVICE) ==
1)).sum().item()
    FP = FP + ((predictions.to(DEVICE) == 1) & (y_batch.to(DEVICE) ==
0)).sum().item()
    FN = FN + ((predictions.to(DEVICE) == 0) & (y_batch.to(DEVICE) ==
1)).sum().item()

    del X_batch
    del y_batch
    del predictions
    gc.collect()
    torch.cuda.empty_cache()

    """
    X_batch.detach()
    X_batch.grad = None

    y_batch.detach()
    y_batch.grad = None

    predictions.detach()
    predictions.grad = None
    """

```

```
#Валидация
```

```
#tensor.detach()
```

```
#tensor.grad = None
```

```
#tensor.storage().resize_(0)
```

```
#подсчет асс
```

```
num_correct_val = 0
```

```
num_samples_val = 0
```

```
#подсчет метрик
```

```
TP_val = 0
```

```
FP_val = 0
```

```
FN_val = 0
```

```
loss_val = 0
```

```
# model.to(DEVICE_CPU)
```

```
gc.collect()
```

```
torch.cuda.empty_cache()
```

```
#model.to(DEVICE)
```

```
    # Optional when not using Model Specific layer
```

```
for iterat, batch in tqdm_notebook(enumerate(test_loader)):
```

```
    #print("iter"+str(iterat)+"\n")
```

```
    model.eval()
```

```
    with torch.no_grad():
```

```
        X_batch, y_batch = batch
```

```

y_prediction = model(X_batch.to(DEVICE))
loss = loss_func(y_prediction,y_batch.to(DEVICE))
#print("Тренировачный лосс \n")
#print(loss.detach().cpu().numpy().item())
#losses_val.append(loss.detach().cpu().numpy().item())
loss_val = loss_val + loss.detach().cpu().numpy().item()
_, predictions = y_prediction.max(1)
#собираем acc
num_correct_val += (predictions == y_batch.to(DEVICE)).sum()
num_samples_val += predictions.size(0)

TP_val = TP_val + ((predictions.to(DEVICE) == 1) &
(y_batch.to(DEVICE) == 1)).sum().item()
FP_val = FP_val + ((predictions.to(DEVICE) == 1) &
(y_batch.to(DEVICE) == 0)).sum().item()
FN_val = FN_val + ((predictions.to(DEVICE) == 0) &
(y_batch.to(DEVICE) == 1)).sum().item()

del X_batch
del y_batch
del predictions
gc.collect()
torch.cuda.empty_cache()
"""

X_batch.detach()
X_batch.grad = None

y_batch.detach()
y_batch.grad = None

```

```

    predictions.detach()
    predictions.grad = None
    """

# Тренировочные

# собираем средний лосс
#total_loss.append(np.mean(losses_train))
total_loss.append(loss_train/len(train_loader))

#считаем acc
acc=float(num_correct)/float(num_samples)

#считаем precision
precision = TP / (TP + FP) if TP + FP > 0 else 0
#считаем recall
recall = TP / (TP + FN) if TP + FN > 0 else 0
#считем f1
f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0
else 0

# собираем acc
total_acc_train.append( acc)
# собираем Precision
total_prec_train.append(precision)
# собираем Recall
total_recall_train.append(recall)
# собираем F1-Score
total_f1_train.append( f1)

```

```

print(f'Валидационный тест датасет')
#print(f'{epoch+1},\t loss: {losses_train[-1]}')
print(f'{epoch+1},\t loss: {loss_train/len(train_loader)}')
print(f'acc: {acc}')
print(f'Precision: {precision}')
print(f'Recall: {recall}')
print(f'F1 Score: {f1}')

# Валидационные

# собираем средний лосс
#total_loss_val.append(np.mean(losses_val))
total_loss_val.append(loss_val/len(test_loader))
#считаем acc
acc=float(num_correct_val)/float(num_samples_val)

#считаем precision
precision = TP_val / (TP_val + FP_val) if TP_val + FP_val > 0 else 0
#считаем recall
recall = TP_val / (TP_val + FN_val) if TP_val + FN_val > 0 else 0
#считаем f1
f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0
else 0

# собираем acc
total_acc_val.append( acc)
# собираем Precision
total_prec_val.append(precision)
# собираем Recall
total_recall_val.append(recall)

```



```
# собираем F1-Score
total_f1_val.append( f1)

print(f'Валидационный датасет')
#print(f'{epoch+1},\t loss: {total_loss_val[-1]}) loss_val/len(test_loader)
print(f'{epoch+1},\t loss: {loss_val/len(test_loader)}')
print(f'acc: {acc}')
print(f'Precision: {precision}')
print(f'Recall: {recall}')
print(f'F1 Score: {f1}')
```

Программный код на языке Python для тестирования системы

```
import torchvision
from PIL import Image
from math import floor, sqrt
import pandas as pd
from scipy import stats

import os

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset
from torchvision import datasets, transforms
# import matplotlib.pyplot as plt
import numpy as np

def rs_test(img, bw=2, bh=2, mask=[1, 0, 0, 1]):
    height, width = img.size

    invert_mask = list(map(lambda x: -x, mask))

    blocks_in_row = floor(width / bw)
    blocks_in_col = floor(height / bh)
    r, g, b = img.split()[:3]
    r = r.load()
    g = g.load()
    b = b.load()
```

```

group_couters = [
    {'R': 0, 'S': 0, 'U': 0, 'mR': 0, 'mS': 0, 'mU': 0, 'iR': 0, 'iS': 0, 'iU': 0, 'imR':
0, 'imS': 0, 'imU': 0},
    {'R': 0, 'S': 0, 'U': 0, 'mR': 0, 'mS': 0, 'mU': 0, 'iR': 0, 'iS': 0, 'iU': 0, 'imR':
0, 'imS': 0, 'imU': 0},
    {'R': 0, 'S': 0, 'U': 0, 'mR': 0, 'mS': 0, 'mU': 0, 'iR': 0, 'iS': 0, 'iU': 0, 'imR':
0, 'imS': 0, 'imU': 0}]

```

```

for y in range(blocks_in_col):
    for x in range(blocks_in_row):
        counter_R = []
        counter_G = []
        counter_B = []
        for v in range(bh):
            for h in range(bw):
                counter_R.append(r[y + v, x + h]) # not vice versa?
                counter_G.append(g[y + v, x + h])
                counter_B.append(b[y + v, x + h])

```

```

group_couters[0][get_group(counter_R, mask)] += 1
group_couters[1][get_group(counter_G, mask)] += 1
group_couters[2][get_group(counter_B, mask)] += 1
group_couters[0]['m' + get_group(counter_R, invert_mask)] += 1
group_couters[1]['m' + get_group(counter_G, invert_mask)] += 1
group_couters[2]['m' + get_group(counter_B, invert_mask)] += 1

```

```

counter_R = lsb_flip(counter_R)
counter_G = lsb_flip(counter_G)
counter_B = lsb_flip(counter_B)

```

```

group_couters[0]['i' + get_group(counter_R, mask)] += 1
group_couters[1]['i' + get_group(counter_G, mask)] += 1
group_couters[2]['i' + get_group(counter_B, mask)] += 1
group_couters[0]['im' + get_group(counter_R, invert_mask)] += 1
group_couters[1]['im' + get_group(counter_G, invert_mask)] += 1
group_couters[2]['im' + get_group(counter_B, invert_mask)] += 1

return (solve(group_couters[0]) + solve(group_couters[1]) +
solve(group_couters[2])) / 3

```

```

def get_group(pix, mask):
    flip_pix = pix[:]

    for i in range(len(mask)):
        if mask[i] == 1:
            flip_pix[i] = flip(pix[i])
        elif mask[i] == -1:
            flip_pix[i] = invert_flip(pix[i])

    d1 = smoothness(pix)
    d2 = smoothness(flip_pix)

    if d1 > d2:
        return 'S'

    if d1 < d2:
        return 'R'

```

```
return 'U'
```

```
def flip(val):  
    if val & 1:  
        return val - 1  
  
    return val + 1
```

```
def invert_flip(val):  
    if val & 1:  
        return val + 1  
  
    return val - 1
```

```
def smoothness(pix):  
    s = 0  
    for i in range(len(pix) - 1):  
        s += abs(pix[i + 1] - pix[i])  
  
    return s
```

```
def lsb_flip(pix):  
    return list(map(lambda x: x ^ 1, pix))
```

```
def solve(groups):
```

```

d0 = groups['R'] - groups['S']
dm0 = groups['mR'] - groups['mS']
d1 = groups['iR'] - groups['iS']
dm1 = groups['imR'] - groups['imS']
d0 = abs(d0)
dm0 = abs(dm0)
d1 = abs(d1)
dm1 = abs(dm1)
a = 2 * (d1 + d0)
b = dm0 - dm1 - d1 - d0 * 3
c = d0 - dm0

```

```

D = b * b - 4 * a * c

```

```

if D < 0:
    return 0

```

```

b *= -1

```

```

if D == 0:
    return (b / 2 / a) / (b / 2 / a - 0.5)

```

```

D = sqrt(D)

```

```

x1 = (b + D) / 2 / a
x2 = (b - D) / 2 / a

```

```

if abs(x1) < abs(x2):
    return x1 / (x1 - 0.5)

```

```

return x2 / (x2 - 0.5)

def model_neuron(model, img_tensor):
    with torch.no_grad():
        scores = model(img_tensor)
        print("Вероятностный ответ scores= ")

        class_prob = torch.softmax(scores, dim=1)

        class_prob, predictions = torch.max(class_prob, 1)
        # _, predictions = scores.max(1)
        print(class_prob)
    return predictions

if __name__ == "__main__":
    # img = Image.open("pure.png")
    # print(rs_method(img))

    # Класс изображения
    classes = ["Empty", "LSB", "Koch_Zhao"]
    # files_class = []

    # Список изображений
    path = "D:\\Program Files\\ml\\System Stega\\dir"
    dir_list = os.listdir(path)
    print(dir_list)

    # Изображения в тензор
    transform = transforms.Compose([
        transforms.ToTensor()

```

```
)
```

```
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
# Загрузка модели
```

```
print("device ")
```

```
print(DEVICE)
```

```
print("\n")
```

```
model = torchvision.models.resnet50(weights="DEFAULT")
```

```
model.classifier = nn.Sequential(
```

```
    nn.Linear(512, 256),
```

```
    nn.ReLU(),
```

```
    nn.Linear(256, 128),
```

```
    nn.ReLU(),
```

```
    nn.Linear(128, 64),
```

```
    nn.ReLU(),
```

```
    nn.Linear(64, 2)
```

```
)
```

```
model.load_state_dict(torch.load('the_best_model7.pth',
```

```
map_location=torch.device('cpu')))
```

```
# model.to(DEVICE)
```

```
print("model\n")
```

```
print(model)
```

```
print("model\n")
```

```
model.eval()
```

```
# Списки для подсчета результатов
```

```
real_class_list = []
```

```
rs_list = []
```

```
cnn_list = []
```



```

result_list = []

# базовые классы
# на основе на названия определяем класс
for i in range(len(dir_list)):
    for j in range(len(classes)):
        if classes[j] in dir_list[i]:
            real_class_list.append(j)
            break

len_for=len(dir_list)
#len_for = 100
for i in range(0, len_for, 1): # len(dir_list)
    image = Image.open("dir\\" + dir_list[i]) # полный путь
    print("name image")
    print(dir_list[i])
    # print("Image"+str(i)+"\n")
    a = 0
    # RS-стегоанализ

    rs_res = rs_test(image)
    print("Результат\n")
    print(rs_res)
    if rs_res > 0.10:
        print("Тут1")
        a = 1
        rs_list.append(a)
    else:
        print("Тут2")
        a = 0

```

```
rs_list.append(a)
```

```
img_tensor = transform(image).unsqueeze(0)
print("tensor = ")
print(img_tensor.size())
print("-----")
b = model_neuron(model, img_tensor)
cnn_list.append(b.item())
print("b= ")
print(b.item())
if a == 1 or b == 1:
    result_list.append(1)
    print("Есть вставка")
else:
    print("Нет вставки")
    result_list.append(0)
```

```
# Списки для проверки результатов
```

```
check_class_list = []
check_empty_list = []
check_rs_list = []
check_cnn_list = []
for i in range(0, len_for, 1):
    if real_class_list[i] == 0 and result_list[i] == 0:
        check_empty_list.append(1)
    else:
        check_empty_list.append(0)

    if real_class_list[i] == 1 and result_list[i] == 1:
```

```

        check_rs_list.append(1)
    else:
        check_rs_list.append(0)

    if real_class_list[i] == 2 and result_list[i] == 1:
        check_cnn_list.append(1)
    else:
        check_cnn_list.append(0)

    if real_class_list[i] != 0 and result_list[i] == 1:
        check_class_list.append(1)
    else:
        if real_class_list[i] == 0 and result_list[i] == 0:
            check_class_list.append(1)
        else:
            check_class_list.append(0)

# Подсчет количества
rs_sum = sum(rs_list)
cnn_sum = sum(cnn_list)
result_sum = sum(result_list)

check_class_sum = sum(check_class_list)
check_empty_sum = sum(check_empty_list)
check_rs_sum = sum(check_rs_list)
check_cnn_sum = sum(check_cnn_list)

print("Вывод результатов\n")
print("rs_sum " + str(rs_sum) + "\n")
print("cnn_sum " + str(cnn_sum) + "\n")

```

```
print("result_sum " + str(result_sum) + "\n")
print("check_class_sum " + str(check_class_sum) + "\n")
print("check_empty_sum " + str(check_empty_sum) + "\n")
print("check_rs_sum " + str(check_rs_sum) + "\n")
print("check_cnn_sum " + str(check_cnn_sum) + "\n")
```

Примеры изображений, использованных во время разработки.



Рисунок 1 – Изображения из датасета.



Рисунок 2 – Изображения: пустой, LSB, Koch, - при тестировании систем.



Рисунок 3 – Изображения с различным шумом.

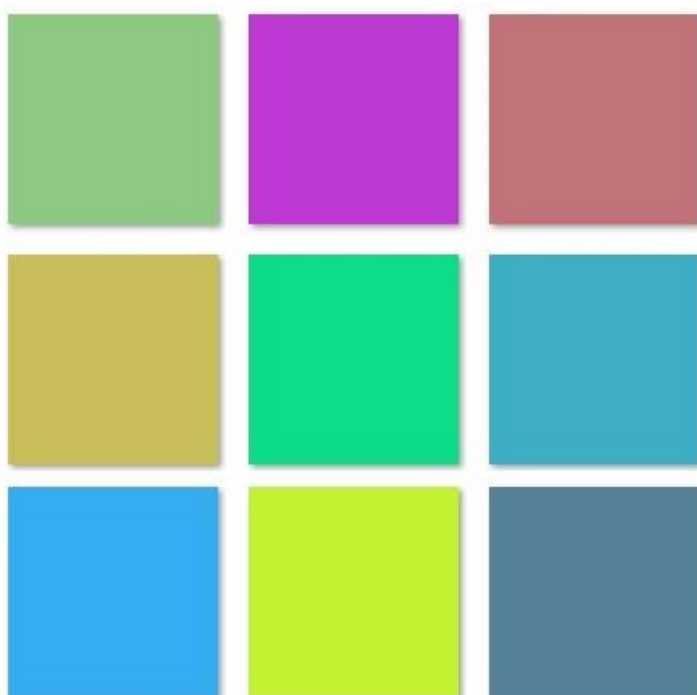


Рисунок 3 – Изображения одноцветные



Рисунок 4 – Изображения с внедренной вставкой Koch в компоненту RGB



Рисунок 5 – Изображения с внедренной вставкой Venham в компоненту RGB и YCbCr.

Усредненные результаты оценки качества, за 10 раз обучения нейронной сети.

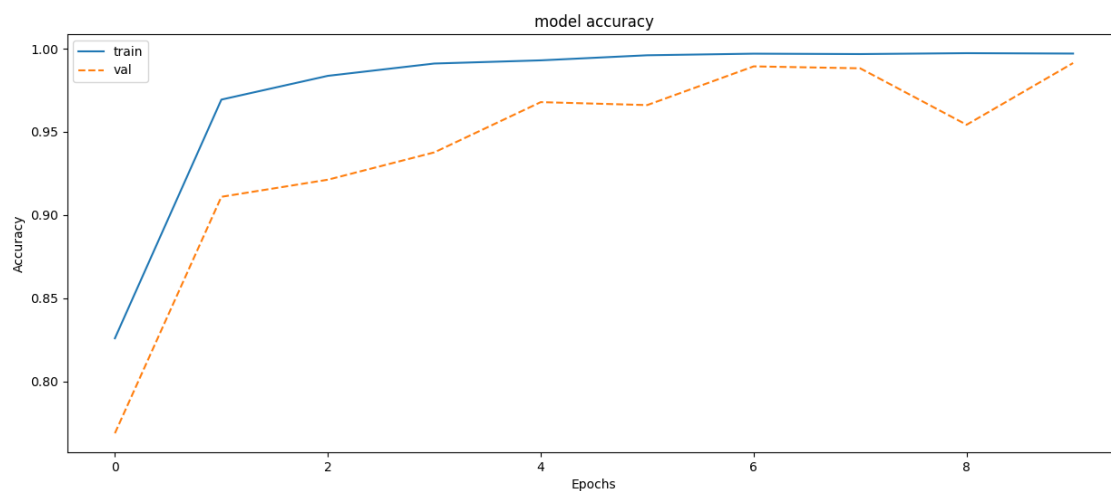


Рисунок 1 - График зависимости значений метрики accuracy от эпохи.

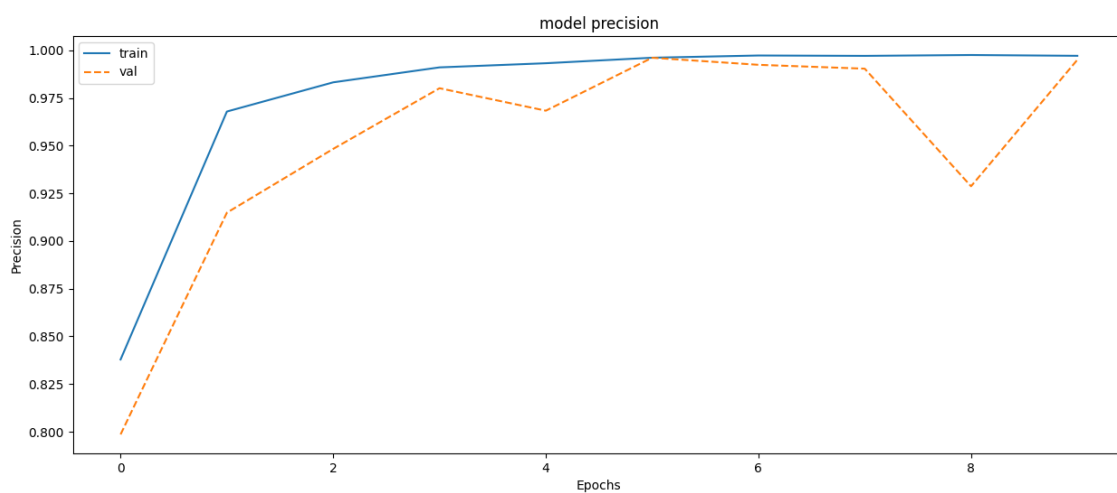


Рисунок 2 - График зависимости значений метрики precision от эпохи.

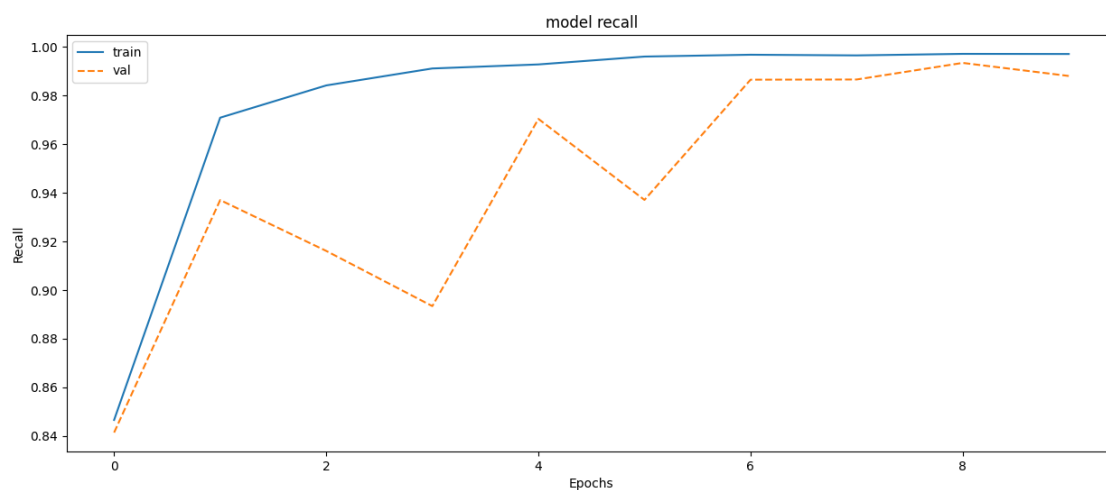


Рисунок 3 - График зависимости значений метрики recall от эпохи.

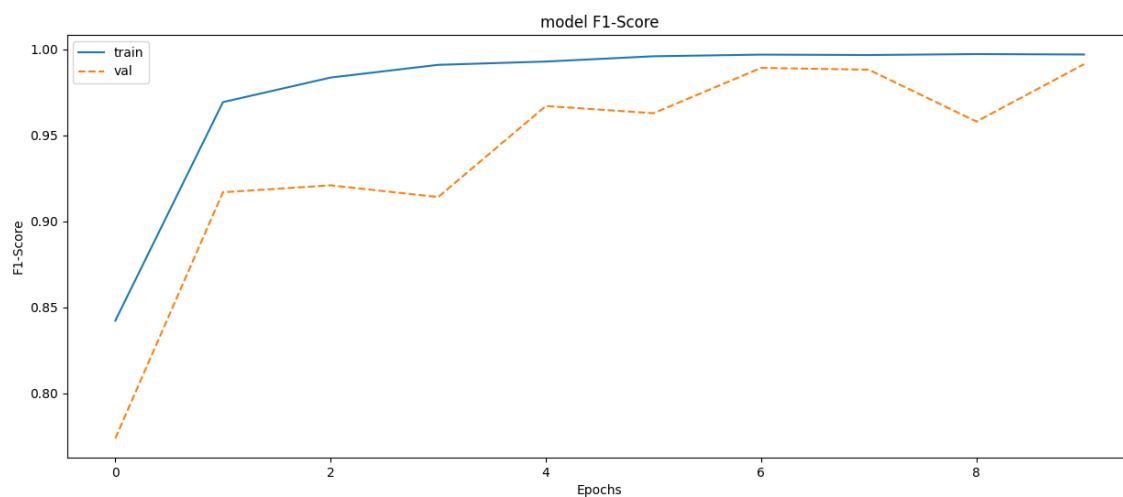


Рисунок 4 - График зависимости значений метрики F-Score от эпохи.

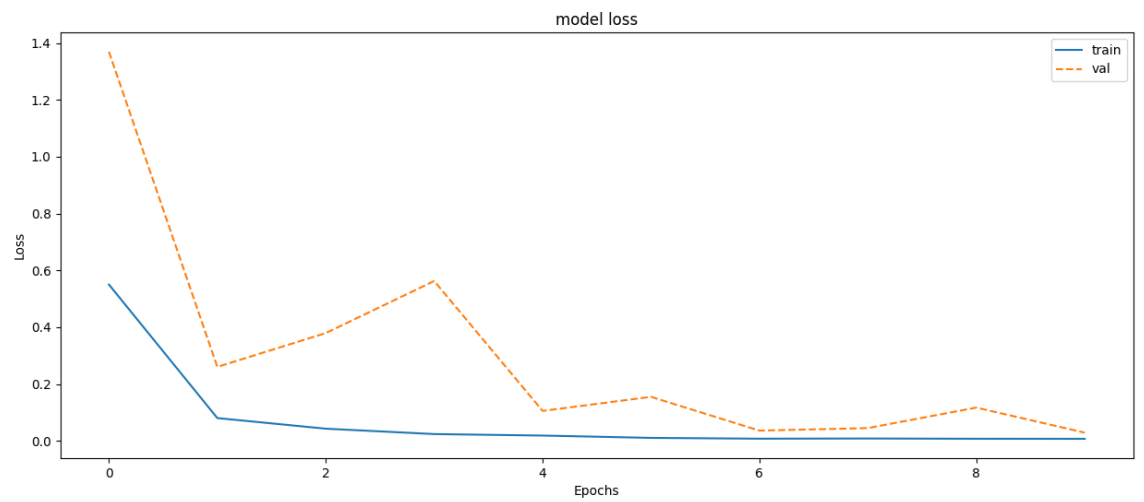


Рисунок 5 - График зависимости значений функции потерь loss от эпохи.